ADVANCED ROBOTIC PERCEPTION / IMAGE PROCESSING AND COMPUTER VISION

MOTION ANALYSIS

RIKKE GADE 2026



AALBORG UNIVERSITY
DENMARK

Agenda

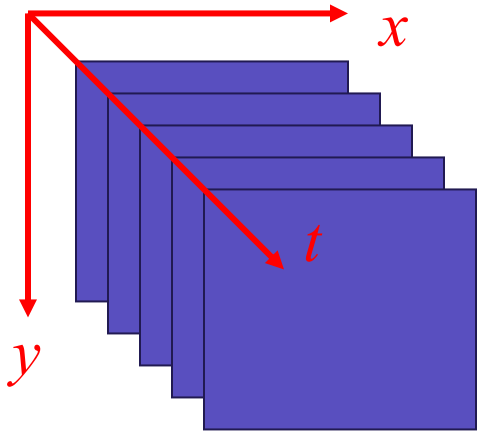
Motion analysis in videos:

- Background subtraction
- Image differencing
- Optical flow



What's a video?

- Videos are image sequences over time



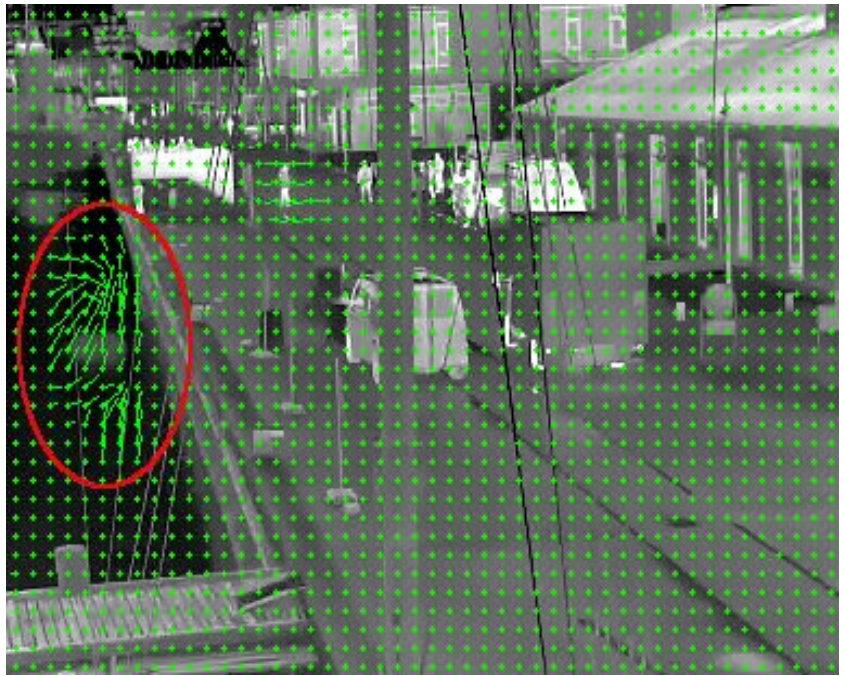
- A video is a function $f(x, y, t)$
- At each time step *t* we have an image $f(x, y)$
- We call each image a **frame**
- Frame rate = the number of images per second
- For example 30 frames/sec (fps)

- All image processing methods apply to video as well: Just process each frame separately!
- But because we now have a sequence of images, we can do more things! ☺

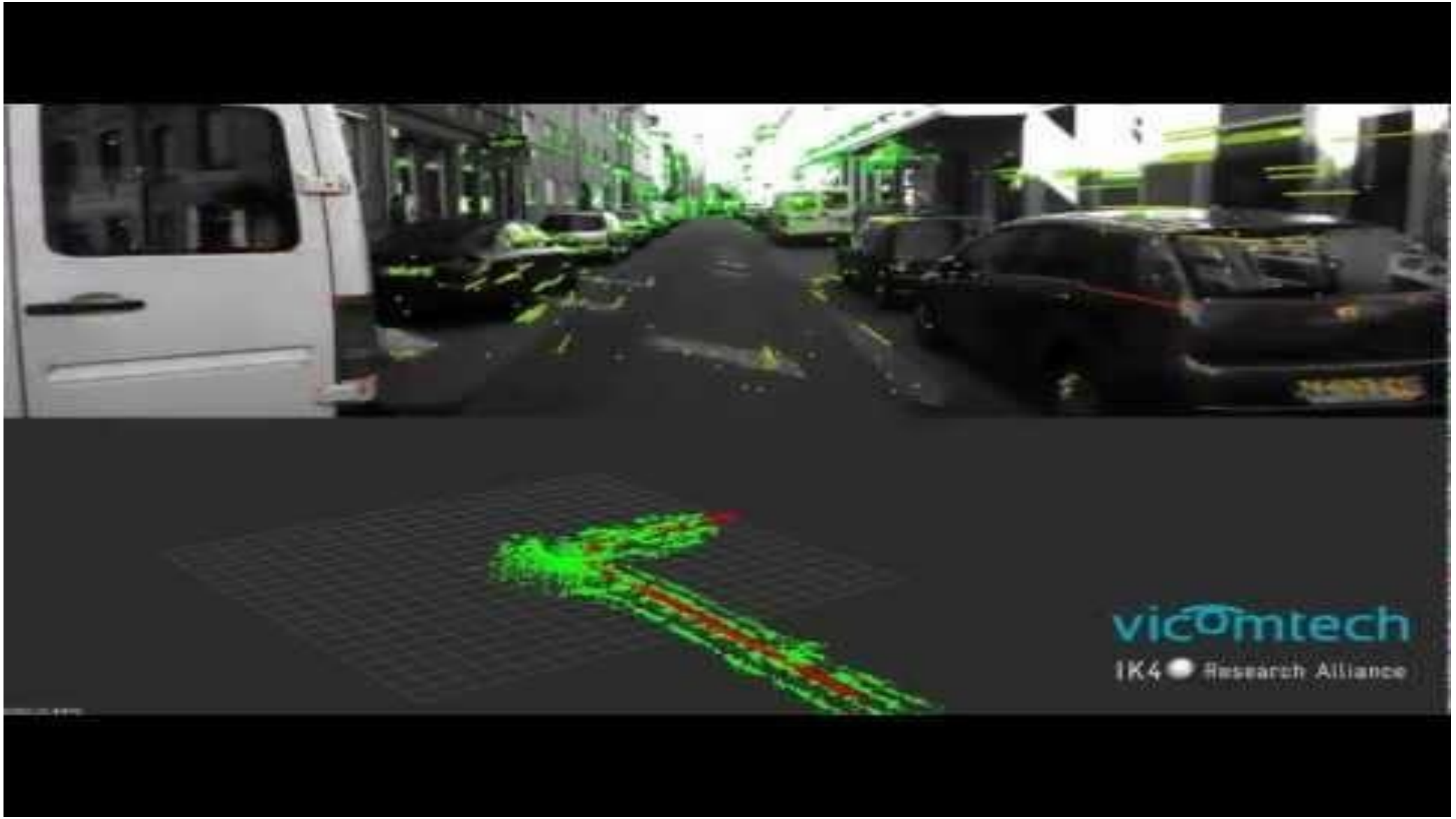
Motion analysis





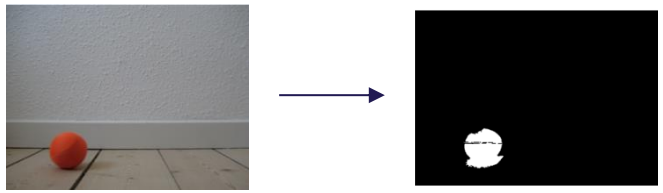


Robotic application of optical flow - Visual Odometry



Segmentation

- Separation of foreground (object) and background (everything else = noise)
- Goal is most often a binary image containing foreground only
- Useful for further processing, such as using silhouettes or analysing the motion of the object.
- Video approach:
  - Motion or changes



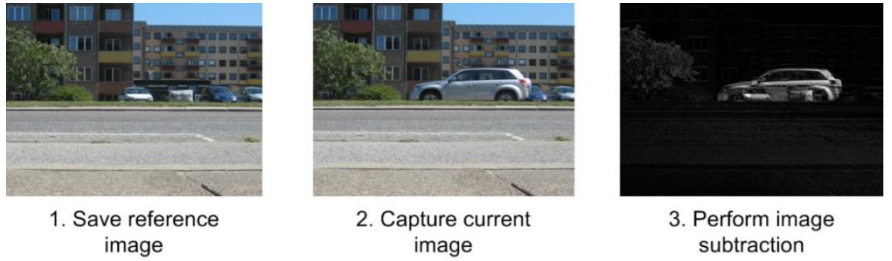
Segmentation in video





Background subtraction

- Assumption: Foreground is moving, background is stable
- Algorithm (simplest version):
  1. Capture image containing background
  2. Capture camera image/new frame
  3. Subtract image (difference = motion)
  4. Threshold (how much difference to be foreground?)
  5. Delete noise



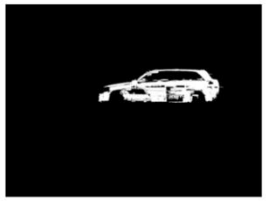
1. Save reference image

2. Capture current image

3. Perform image subtraction



4. Thresholding



5. Filter noise

Background subtraction

- Subtract image
  - Compute pixel-wise
  - Subtract previous image from input image:

$$F(x,y) = I(x,y) - B(x,y)$$






- Usually the absolute difference is applied $|F(x,y)|$:

Background subtraction

- Threshold
  - Decide when a pixel is supposed to be considered as a background pixel, or when it is to be considered as a foreground pixel:

  - Pixel (x,y) is foreground pixel if
    $$|F(x,y)| > TH$$
  - Pixel (x,y) is background pixel if
    $$|F(x,y)| < TH$$

Problem: Which threshold to choose??

1. Calculate average background image
2. Capture camera image
3. Subtract image
4. **Threshold**
5. Delete noise

AALBORG UNIVERSITY
DENMARK

Background subtraction

- Deleting noise
  - Single pixels are likely to appear:
    - Pixel noise!
  - Apply median filter
    - Depending on filter size, bigger spots can be erased
  - Alternative: Morphology



1. Calculate average background image
2. Capture camera image
3. Subtract image
4. Threshold
5. **Delete noise**

Background subtraction

- Example using OpenCV



Input



Background subtraction + median filter

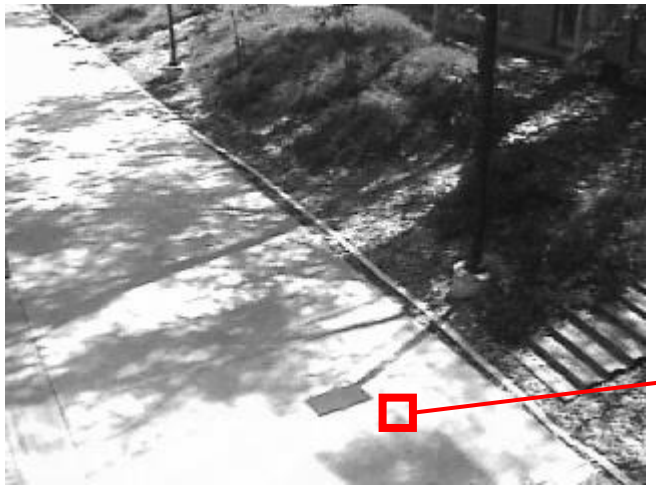
Advanced background subtraction

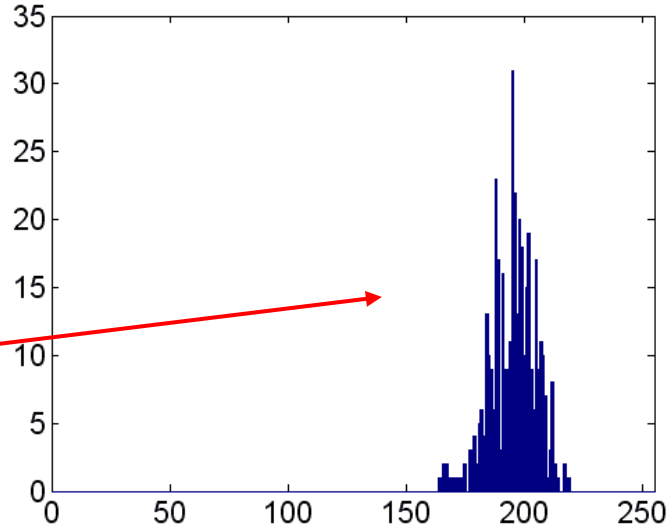
- What if we have small motion in the background?
  - Noise in the camera/lighting

- New algorithm:
  1. **Calculate average background image**
  2. Capture camera image
  3. Subtract image (difference = motion)
  4. Threshold
  5. Delete noise

Advanced background subtraction

- With non-stable background
  - E.g. camera noise, changes in light, small motion from wind gusts
- Learn the background



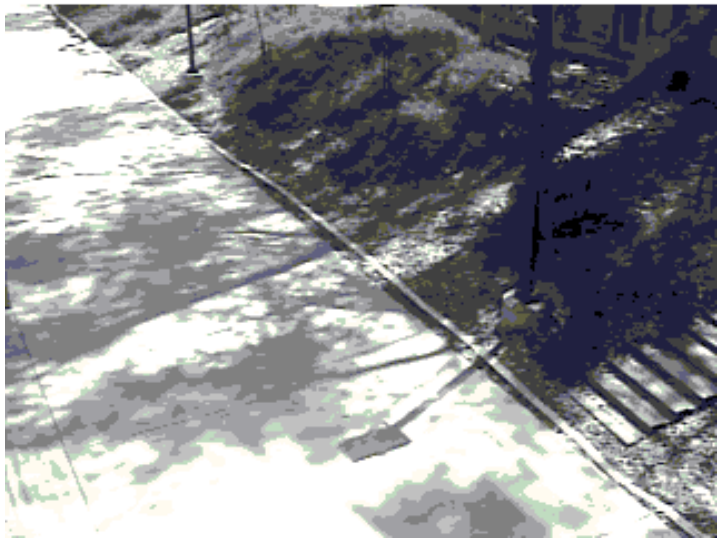


Histogram of N values of pixel (x,y)

Advanced background subtraction

- Learning the background
  - Capture N empty images and calculate the average background image

$$B(x,y) = \sum_{i=1}^{N} I_i(x,y)/N$$



1. **Calculate average background image**
2. Capture camera image
3. Subtract image
4. Threshold
5. Delete noise

Advanced background subtraction

- Global vs. local thresholds

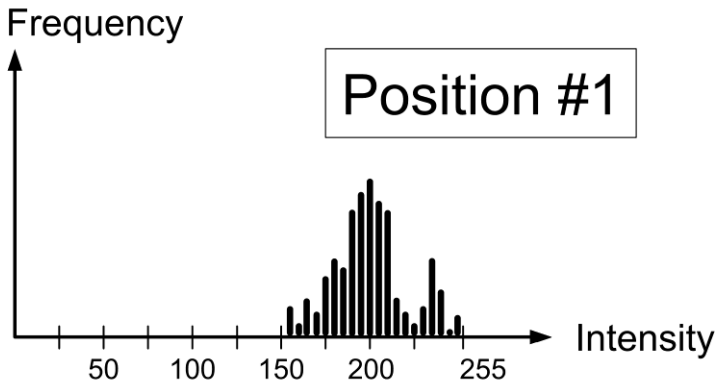
- So far:
  - Image subtraction
    - $F(x,y) = |I(x,y) - B(x,y)|$
  - Global threshold T to obtain binary image bw
    - $bw(x,y) = \begin{cases} 0, & if\ F(x,y) < T \\ 255, & else \end{cases}$

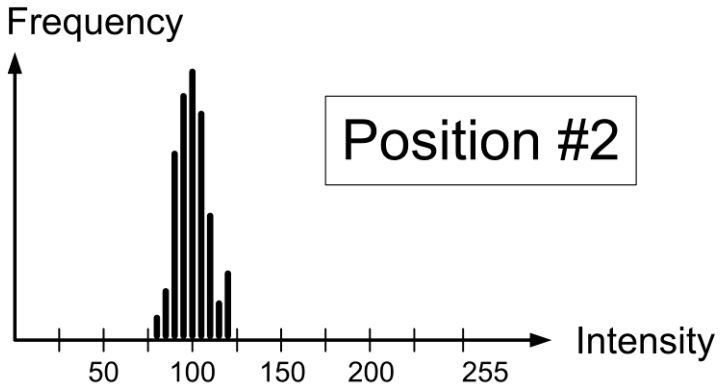
1. Calculate average background image
2. Capture camera image
3. **Subtract image**
4. **Threshold**
5. Delete noise

Very advanced background subtraction

- Problem: TH should be different for each pixel





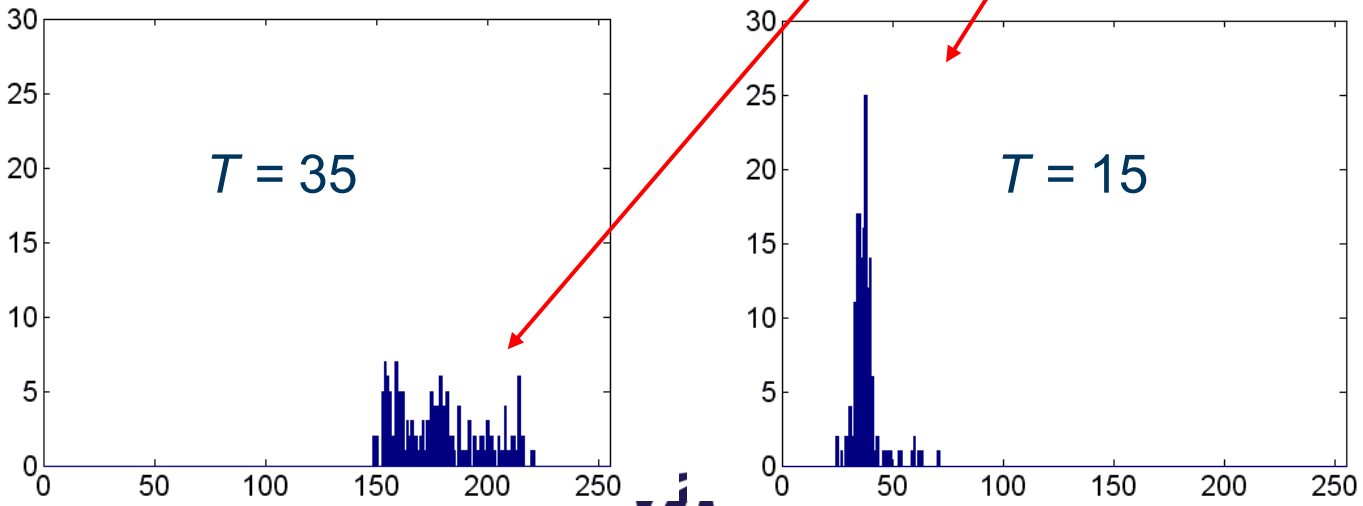


AALBORG UNIVERSITY
DENMARK

Very advanced background subtraction

- If you have local pixel variation
- Then apply local thresholds





Very advanced background subtraction



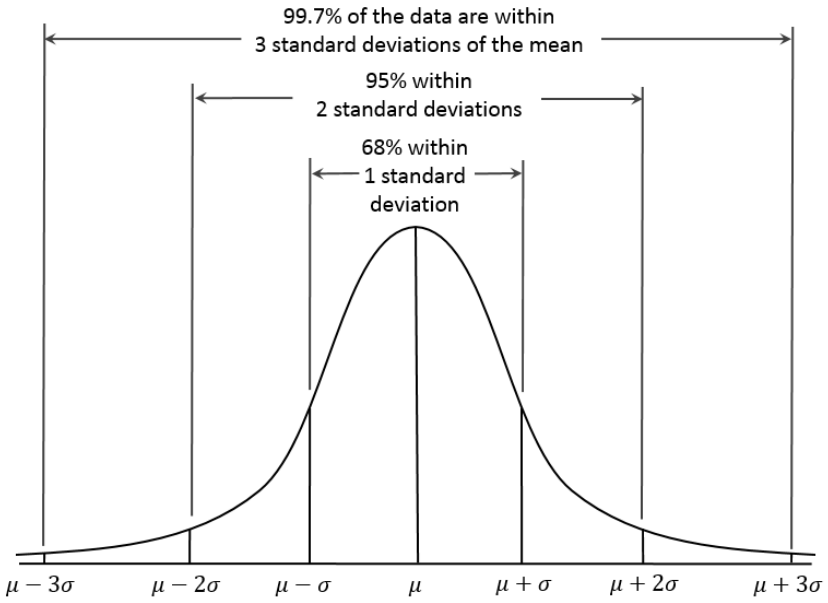
- Learn the background at its variations!
  - Gaussian models (mean and variance) for each pixel
- The more images you train on – the better!

- Idea:
  - Some pixels may vary more than others
- Algorithm:
  - Consider each pixel (x,y) in the new frame and check how much it varies with respect to the mean and variance of the learned Gaussian models?

1. **Calculate mean and variance for each pixel**
2. Capture camera image
3. Subtract mean image
4. **Threshold according to variance**
5. Delete noise

Very advanced background subtraction

- Threshold can be chosen depending on the variance
  - A local threshold
- Standard deviation σ
- For example:
  - If $\left|F_{(x,y)}\right| < TH \Rightarrow$ background pixel
  - TH = K· $\sigma(x,y)$
  - K is set by you
  - K is the same for all pixels



Updating the background image

- If your system has to operate over a long period of time
  - Gradual changes during processing, e.g., light changes during the day
- Then update the reference image



Input



Binary output: Using first frame as background – no update

Updating the background image

- If your system has to operate over a long period of time
  - Light changes during the day, etc.
- Then update the reference image
  - Update speed α
  - $B_{new}(x, y) = \alpha \cdot B_{old}(x, y) + (1 - \alpha) \cdot I(x, y)$



Input



Binary output: Update speed α=0.98

Image differencing

- With very dynamic scenes which can not be learned
- If we assume that only the object is moving → motion can be used to find the object
- Then choose the previous frame as the reference frame

- Use the previous frame as reference frame
  - $R_t(x, y) = I_{(t-1)}(x, y)$

- Different reference image (depending on application)
  - $R_t(x, y) = I_{(t-k)}(x, y)$ $\quad k = 2,3, \ldots$

- Image differencing
  - $F_t(x, y) = abs(I_t(x, y) - R_t(x, y))$

- Double differencing
  - $bw_t(x, y) = bw_{R_1}$ AND bw_{R_2}



AALBORG UNIVERSITY
DENMARK

Image differencing

Ghosts:

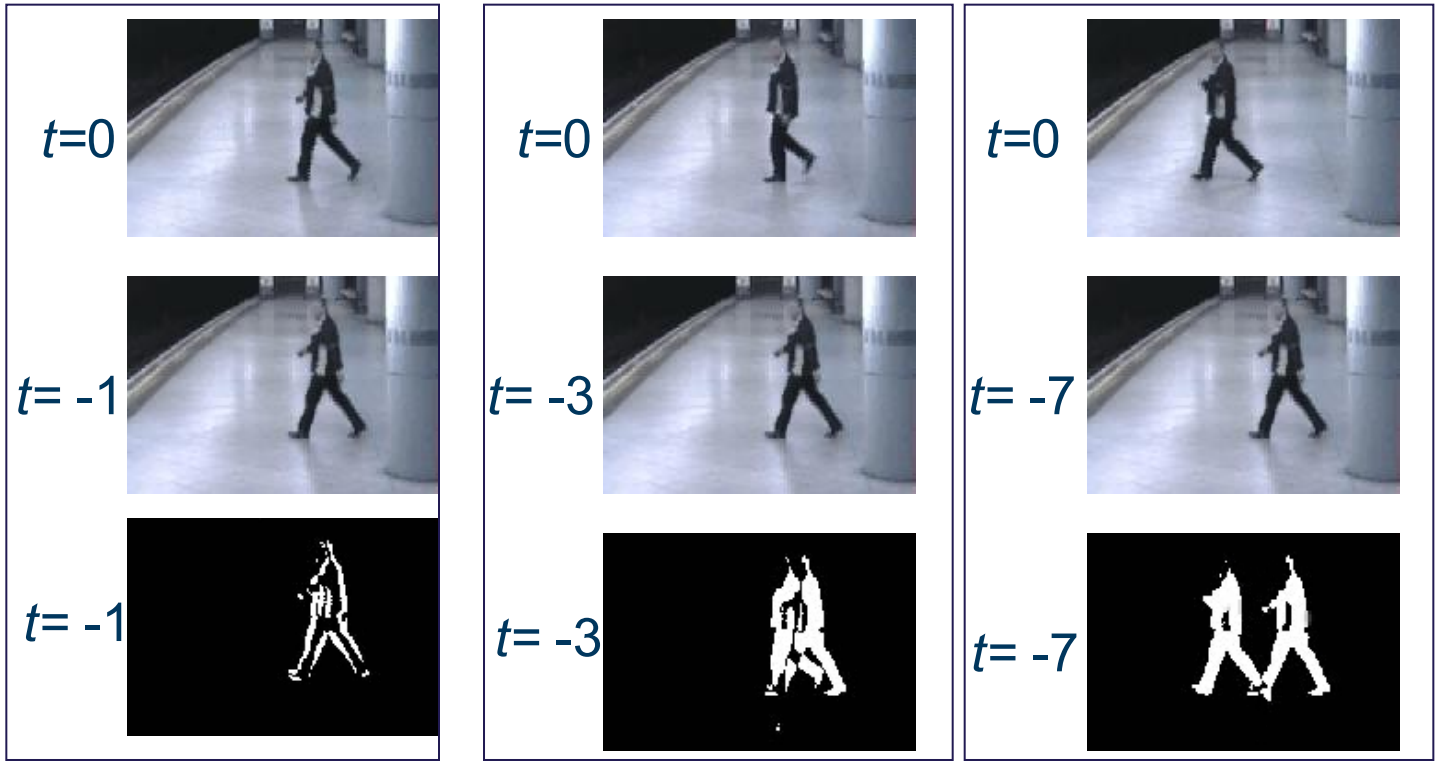


Image differencing vs. background subtraction

Image differencing	Background subtraction
• Handles dynamic scenes • Detects only motion • Detection results in ghosts	• Requires static background • Detects stationary objects • Detects whole objects

OPTICAL FLOW

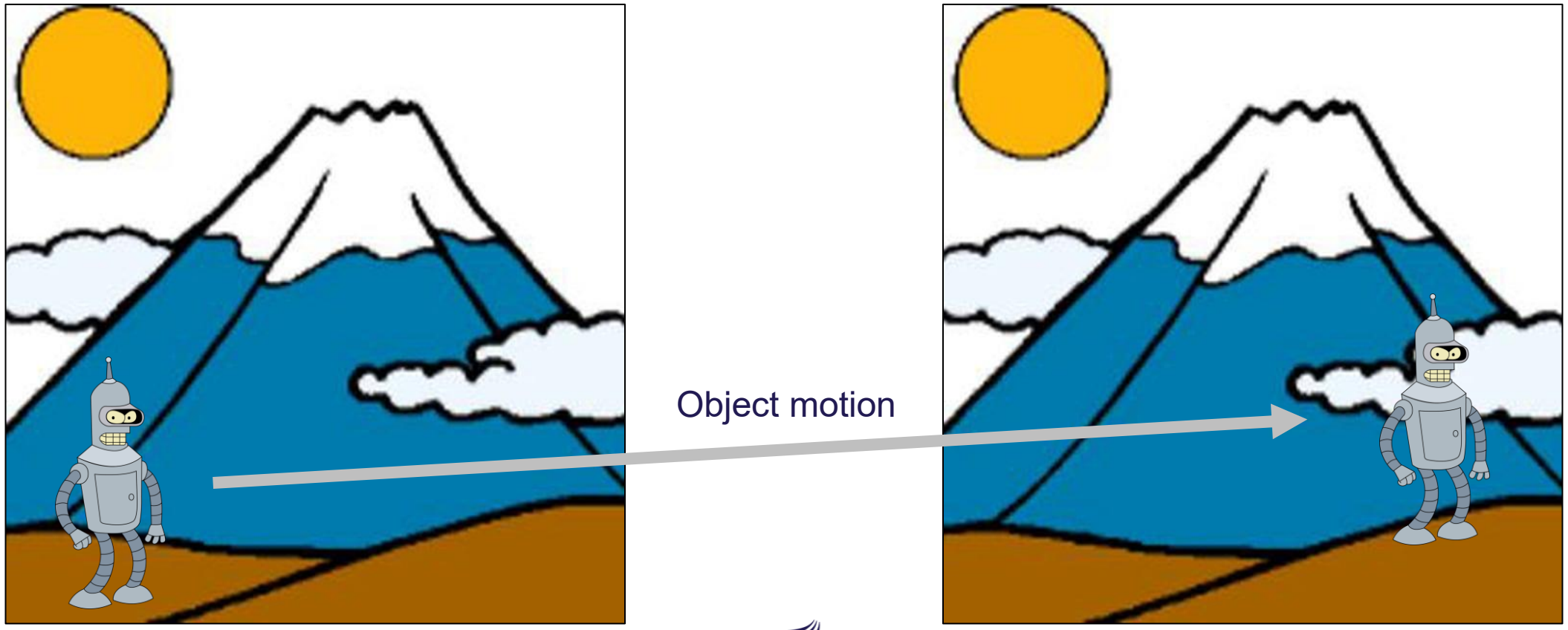
Credits: Slides are adapted from material from University of Wahington, course given by Joseph Redmon, Ali Farhadi, and Connor Schenck. https://pjreddie.com/courses/computer-vision/



AALBORG UNIVERSITY
DENMARK

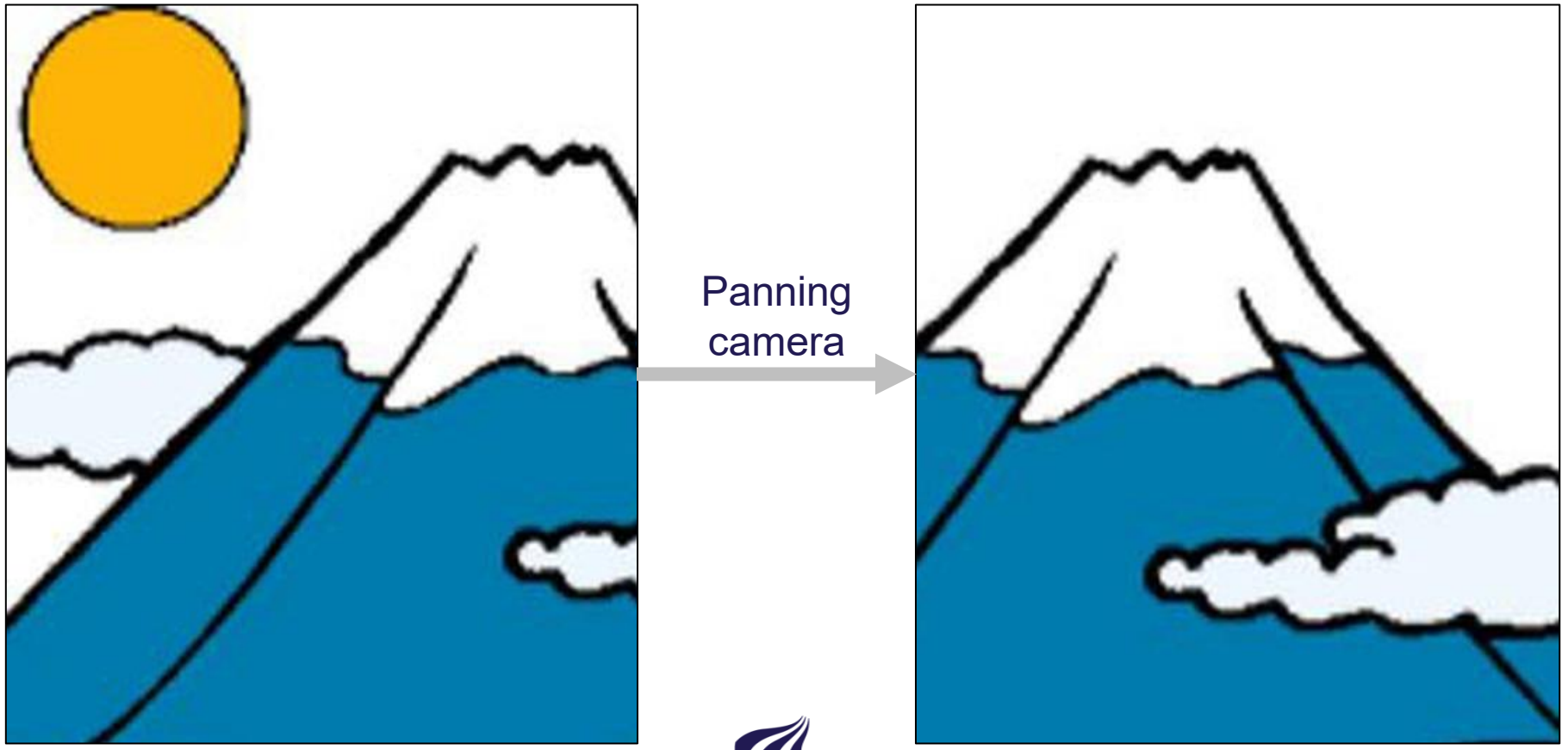
Optical flow

- Looking for motion!



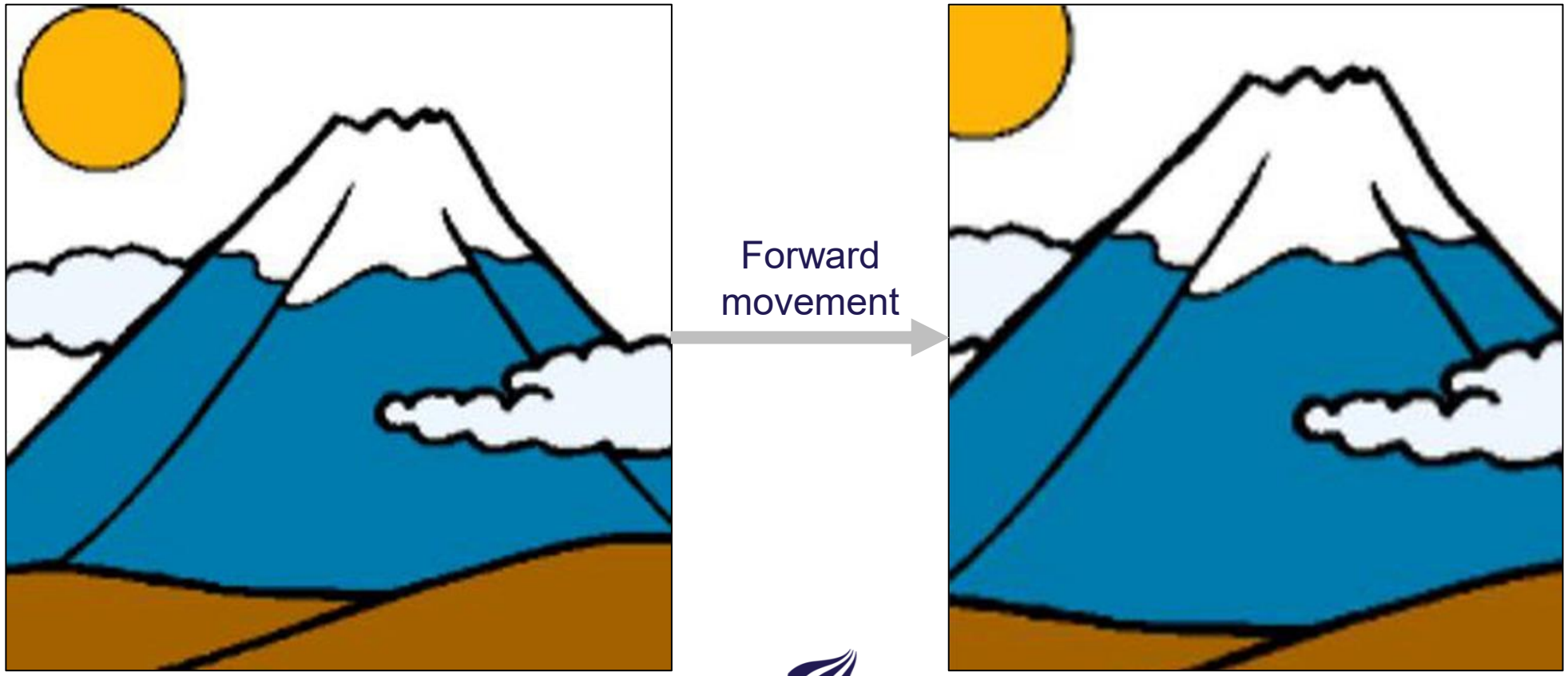
Optical flow

- Looking for motion!

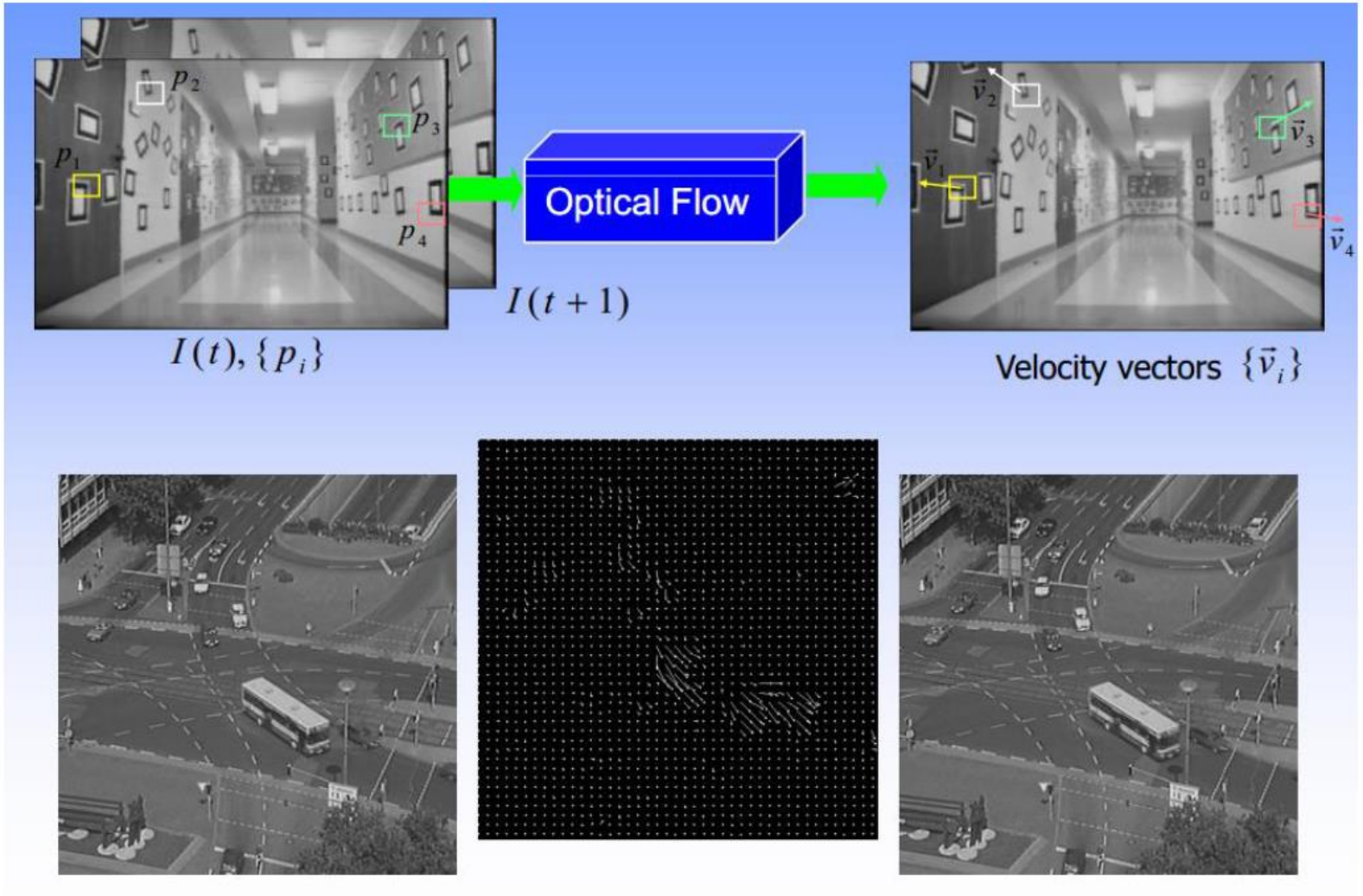


Optical flow

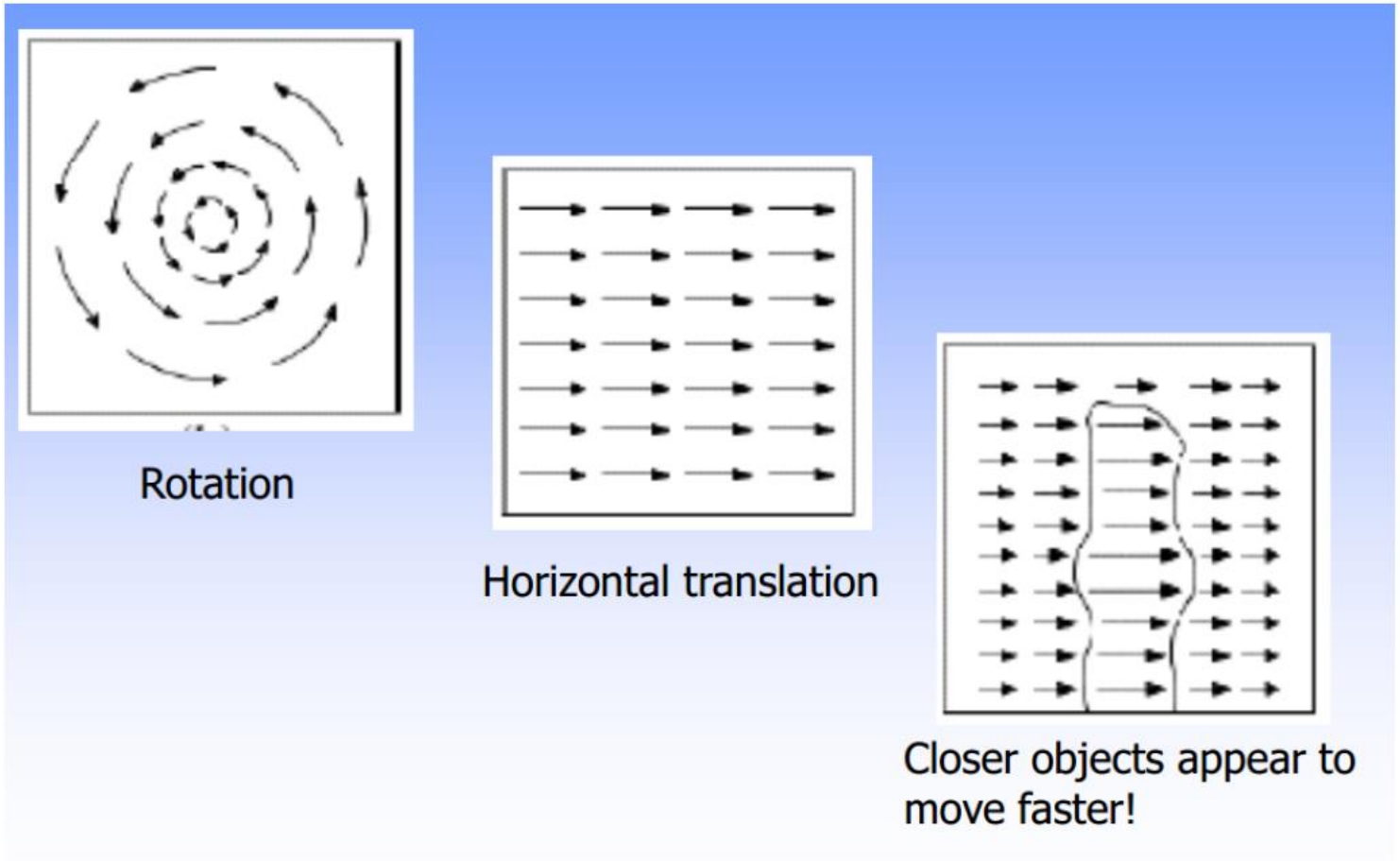
- Looking for motion!



Optical flow



Optical flow

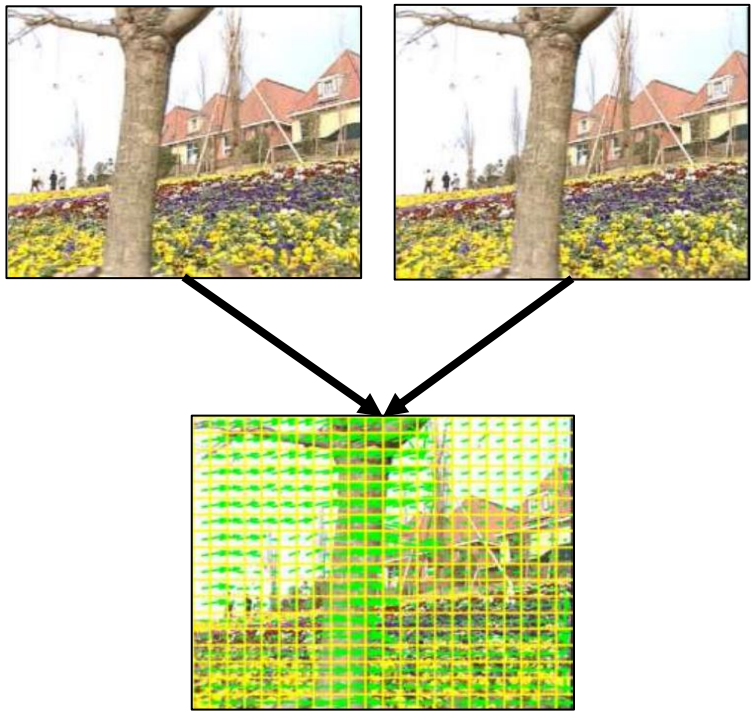


Optical flow - applications

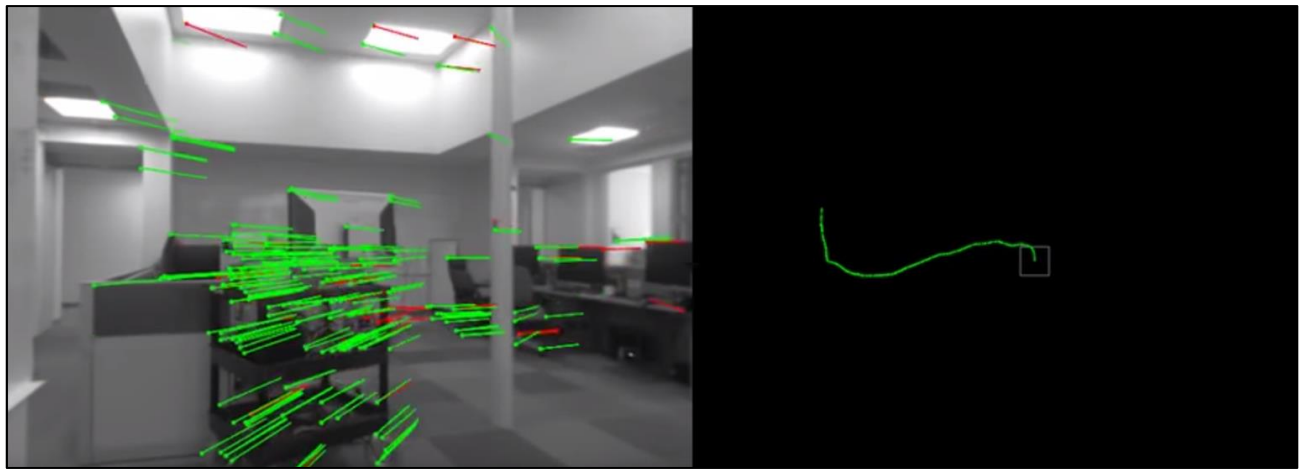
Object Tracking



Motion Estimation

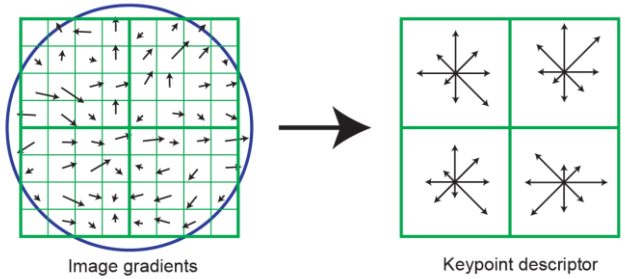


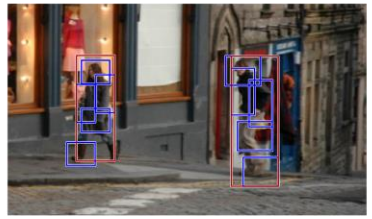
Visual Odometry

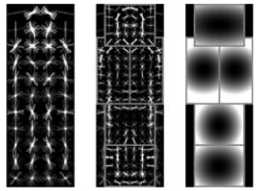


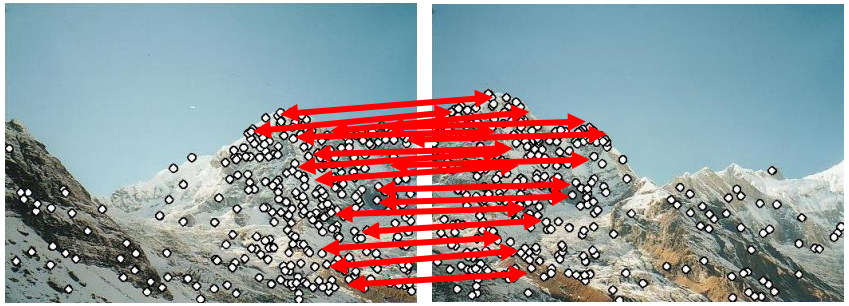
Feature matching

- How to find flow in an image?
  - Feature matching?







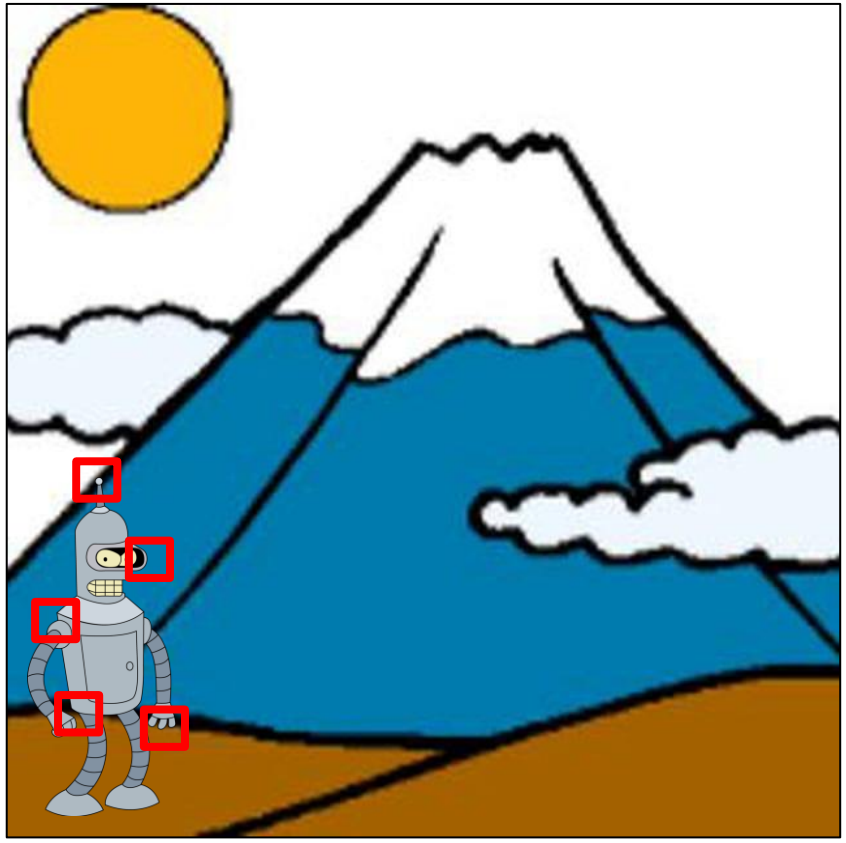


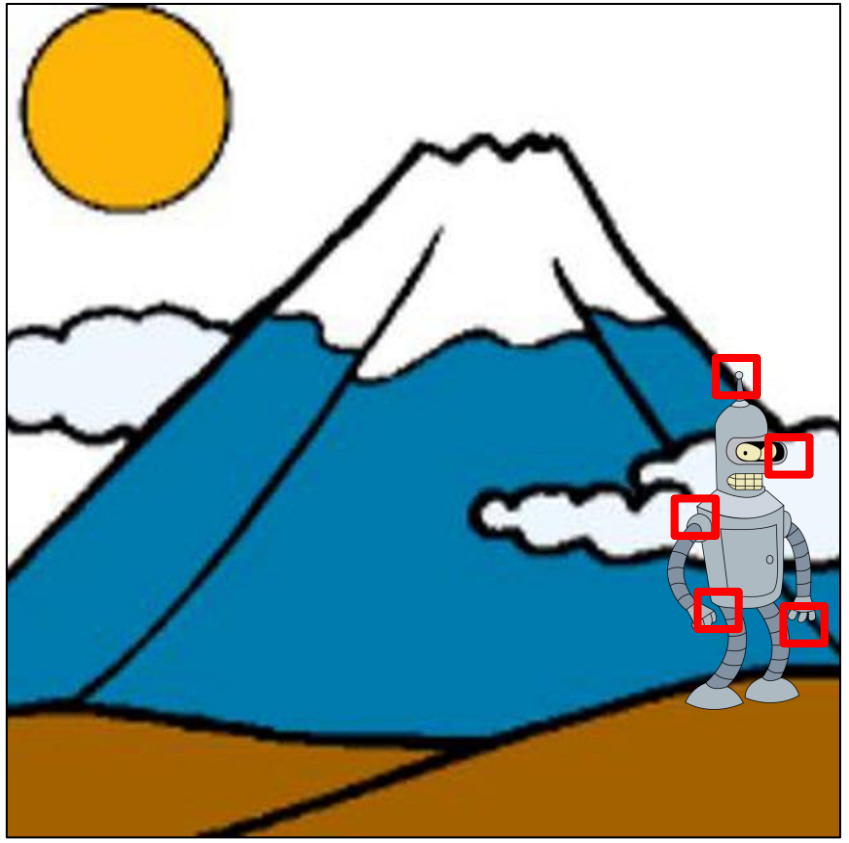




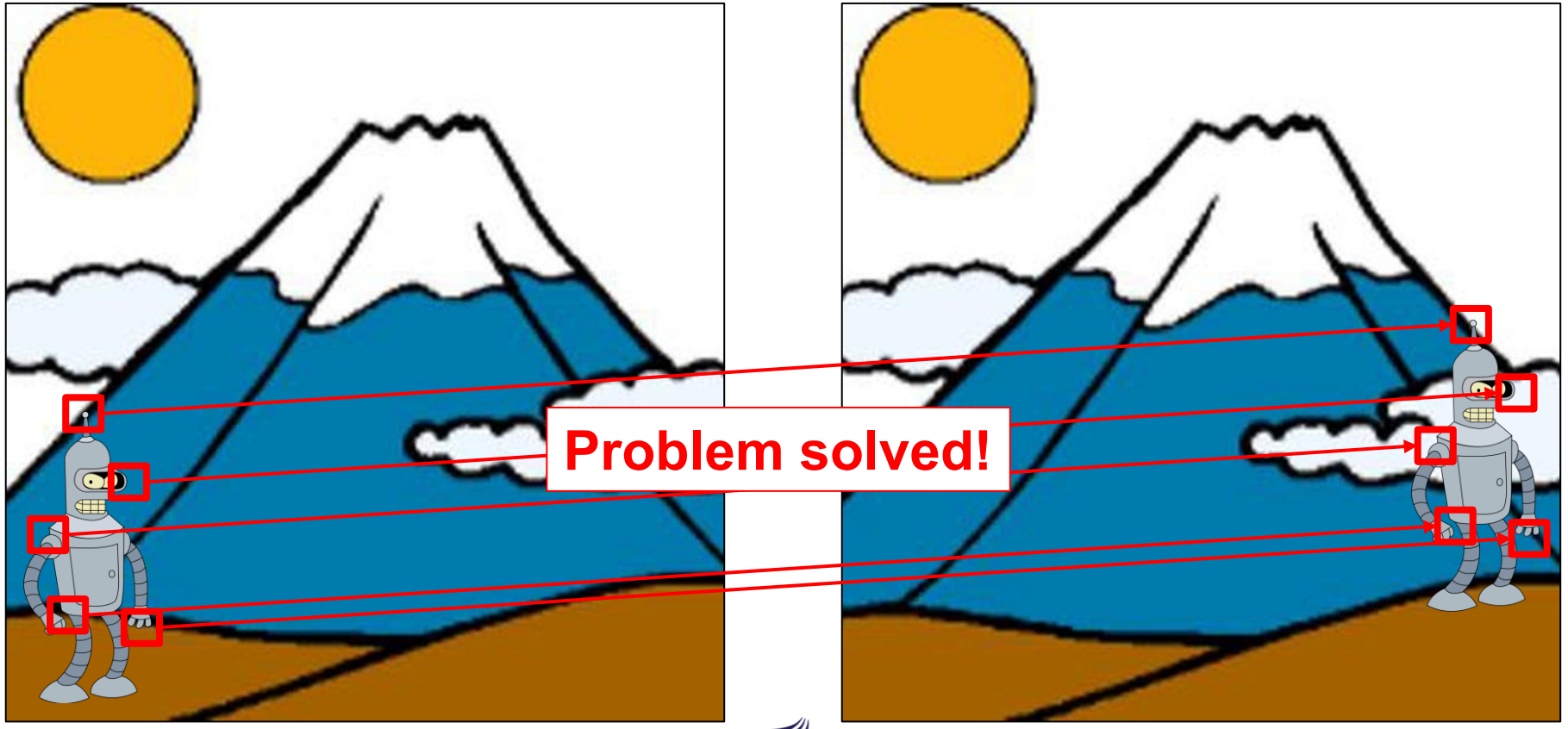


Feature matching





Feature matching



Feature matching

Advantages:

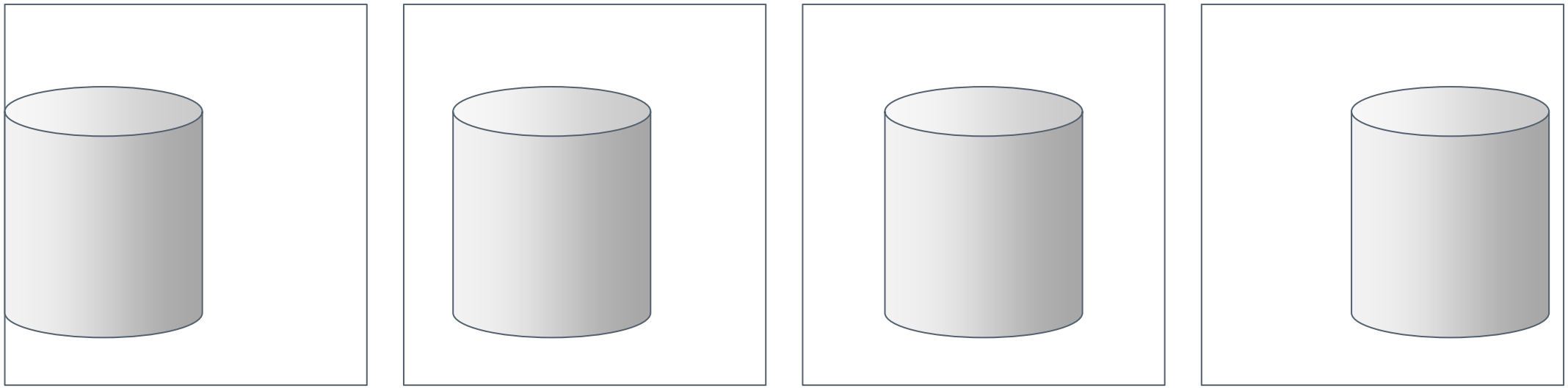
- Scale/rotation invariant
- (Somewhat) lighting invariant
- Can handle large movements

Disadvantages:

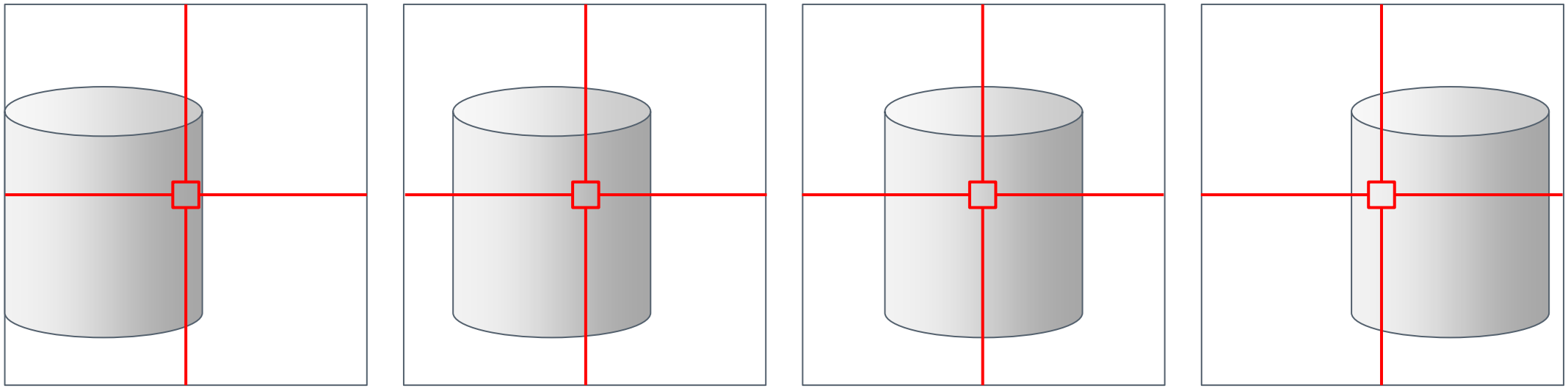
- Sparse
- Feature alignment not exact
- Low accuracy

Not perfect for optical flow!

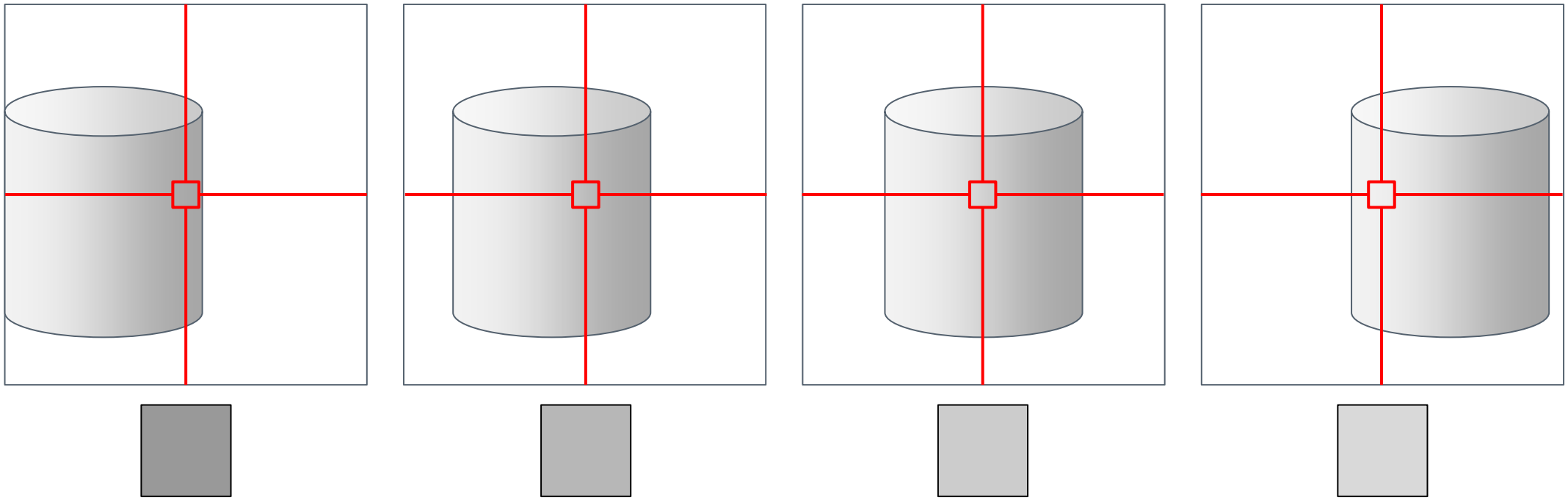
Optical flow - concept



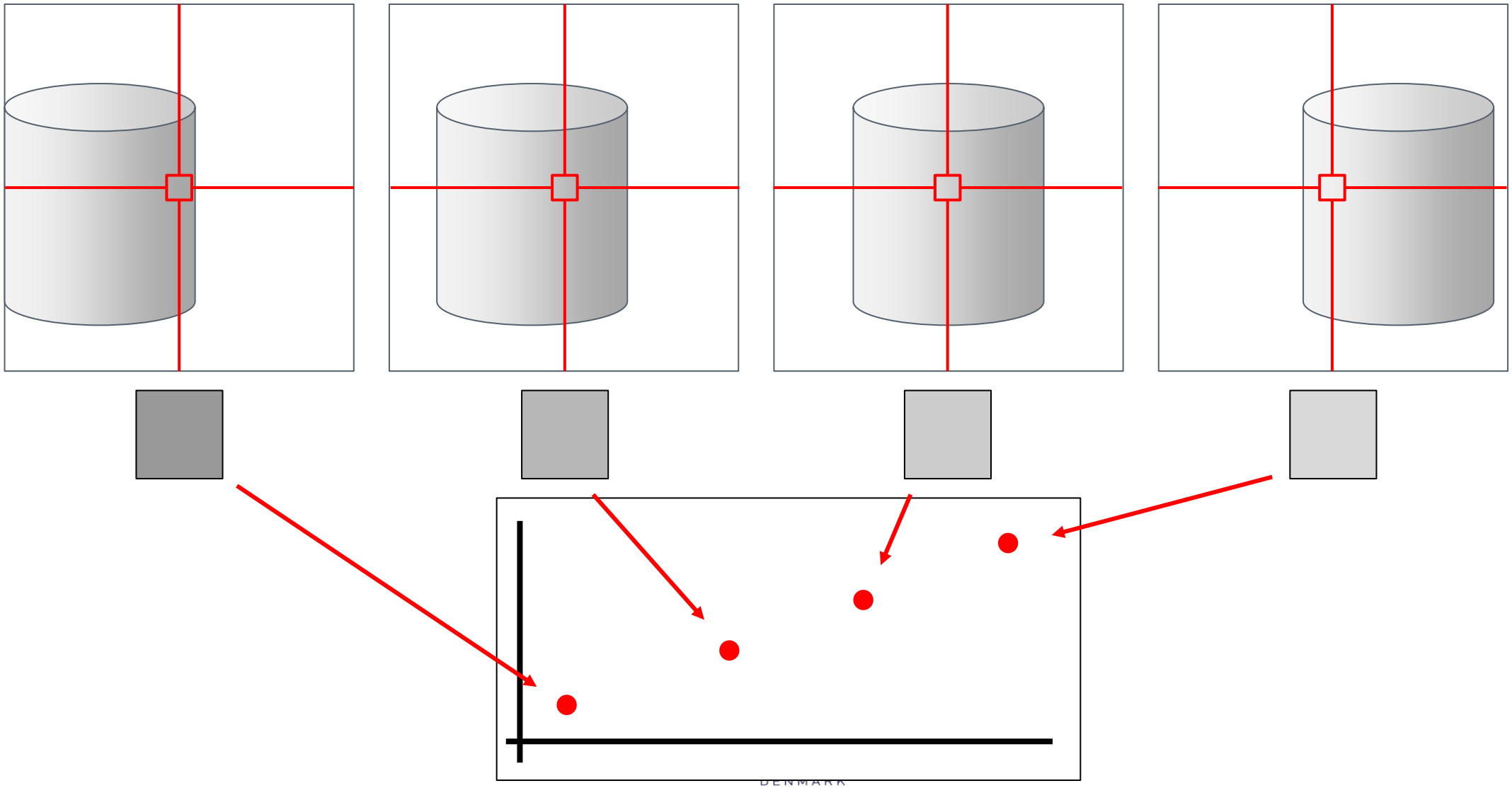
Optical flow - concept



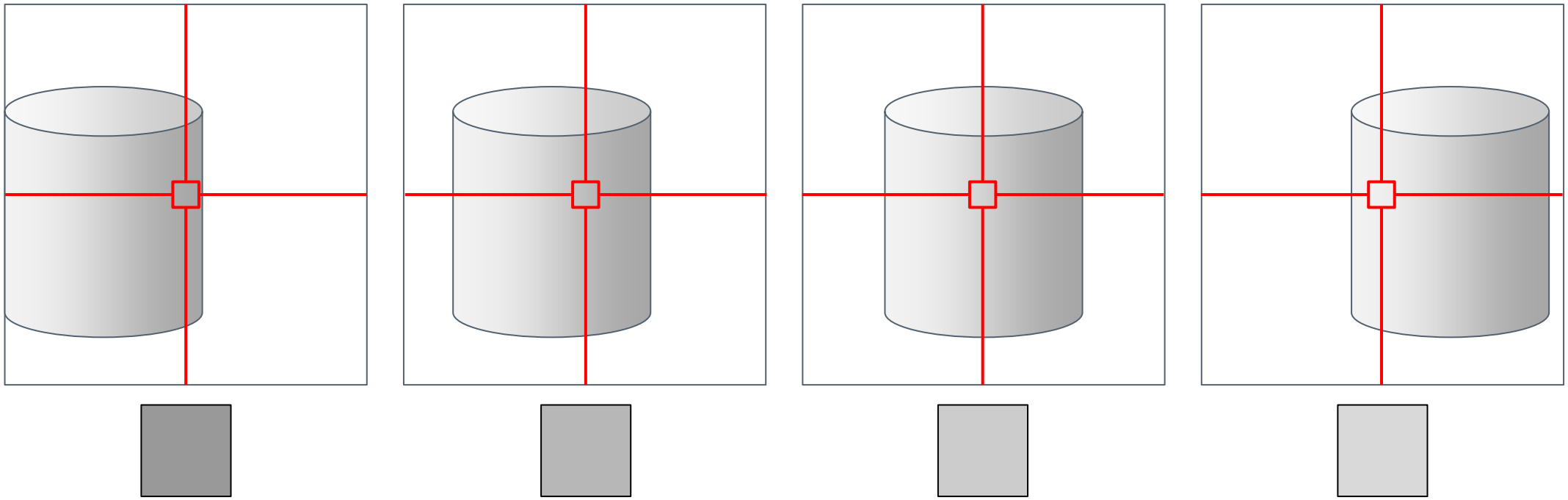
Optical flow - concept



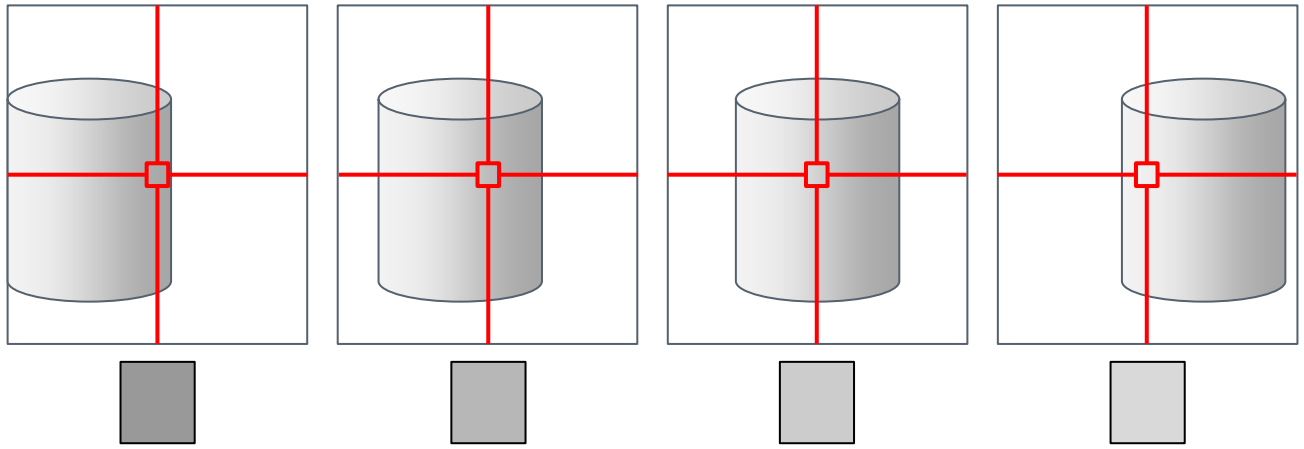
Optical flow - concept

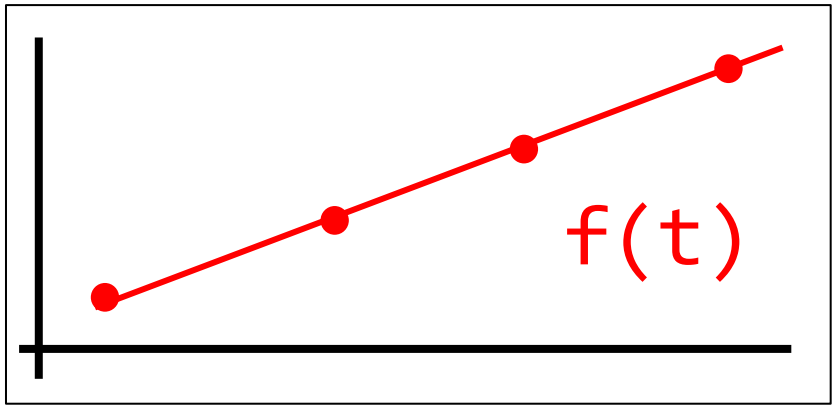


Optical flow - concept

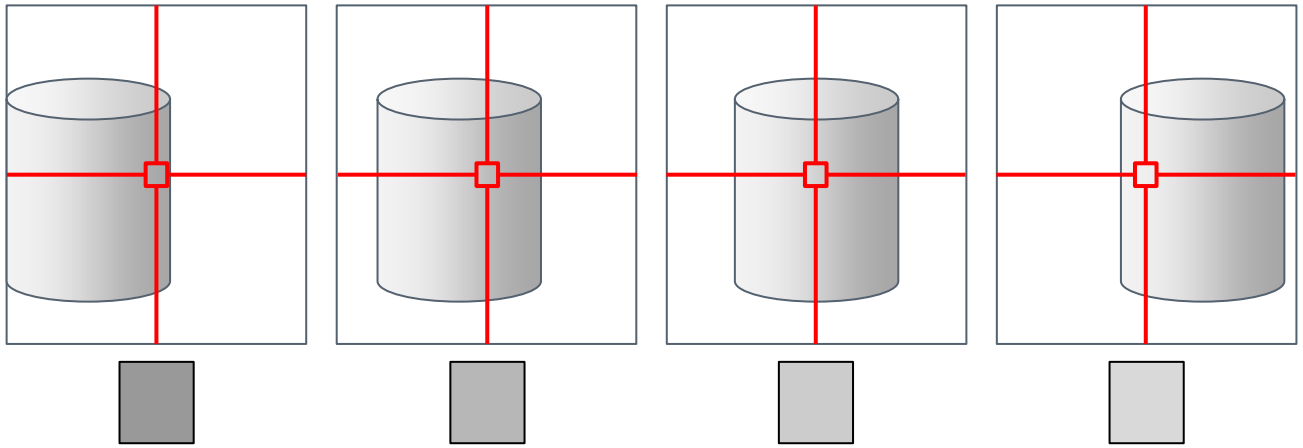


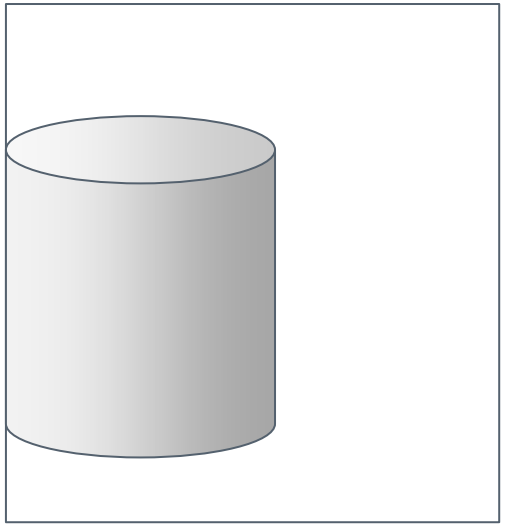
Optical flow - concept

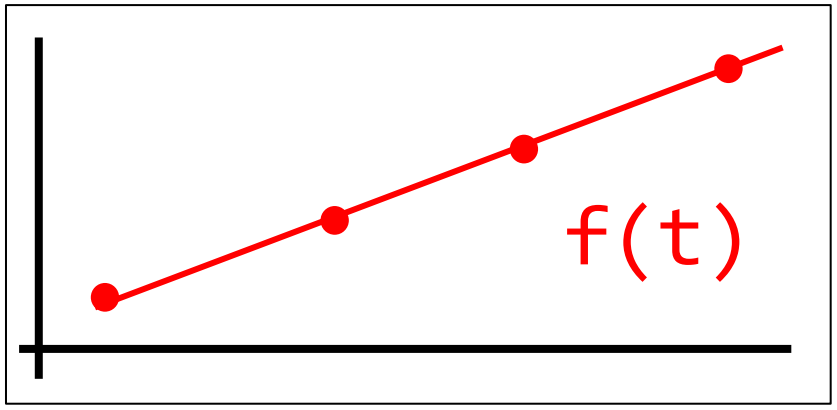




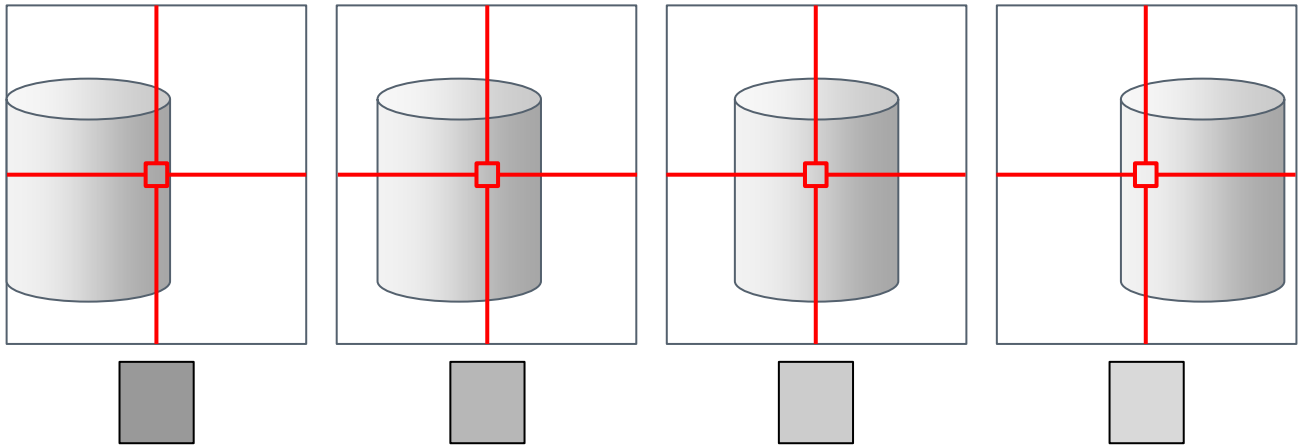
Optical flow - concept

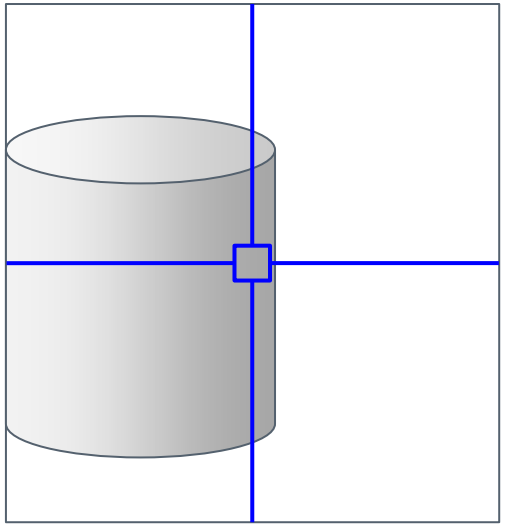


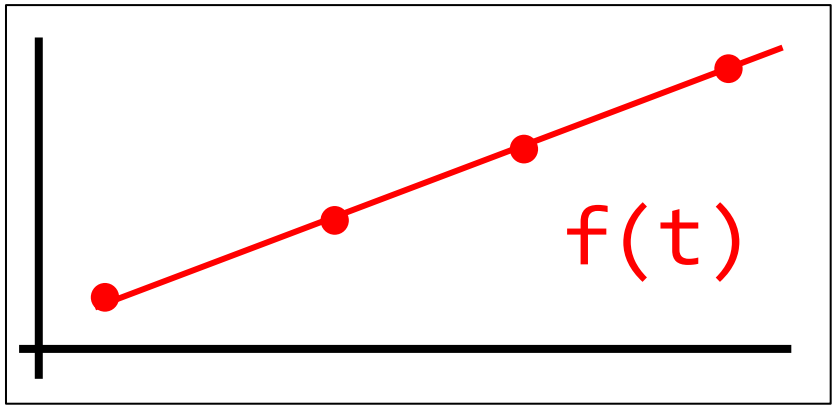




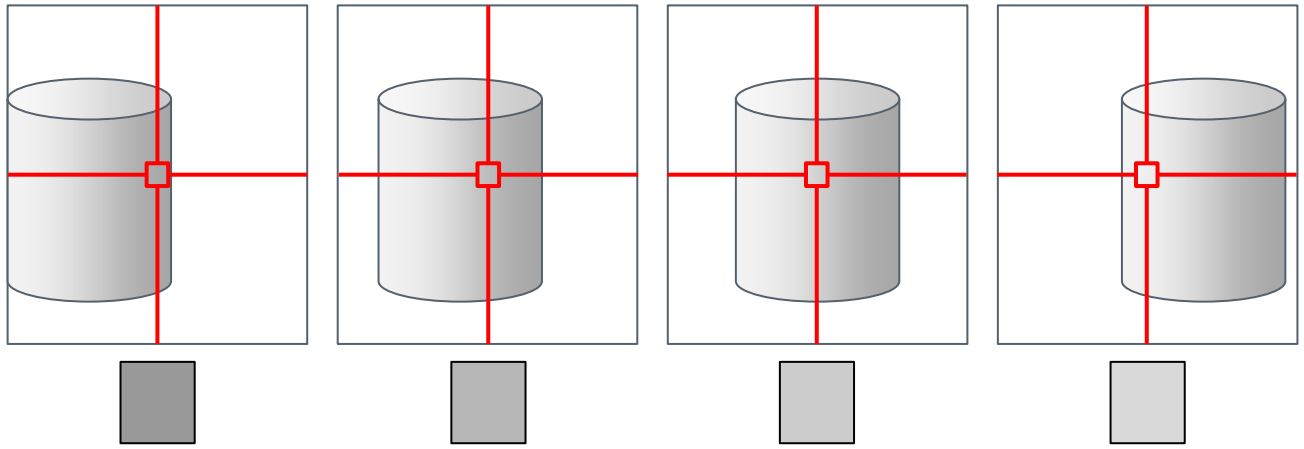
Optical flow - concept

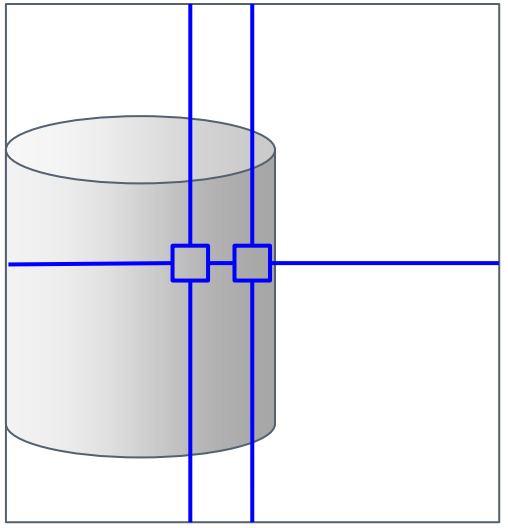


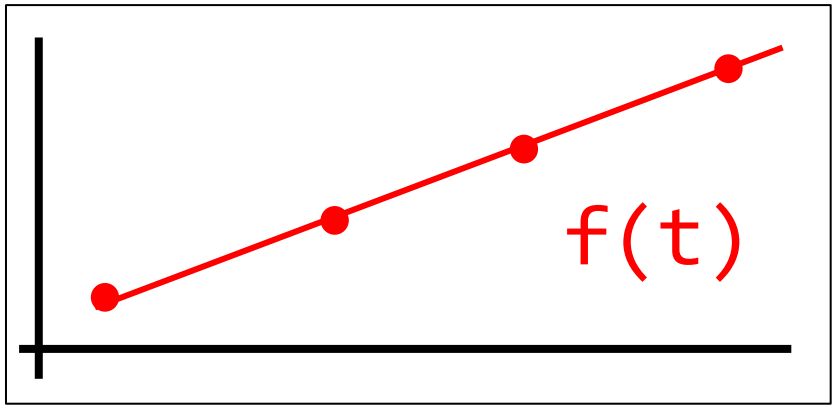




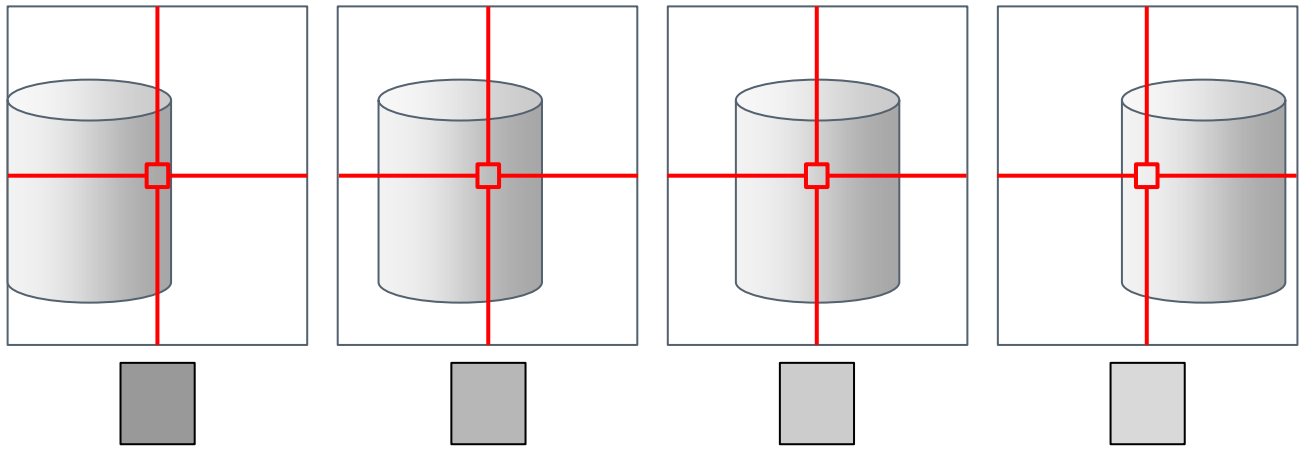
Optical flow - concept

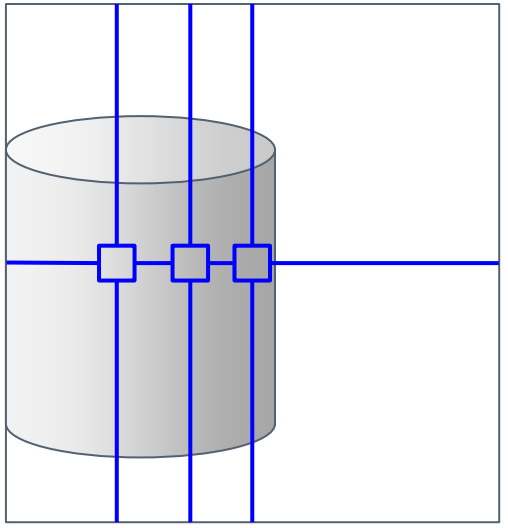


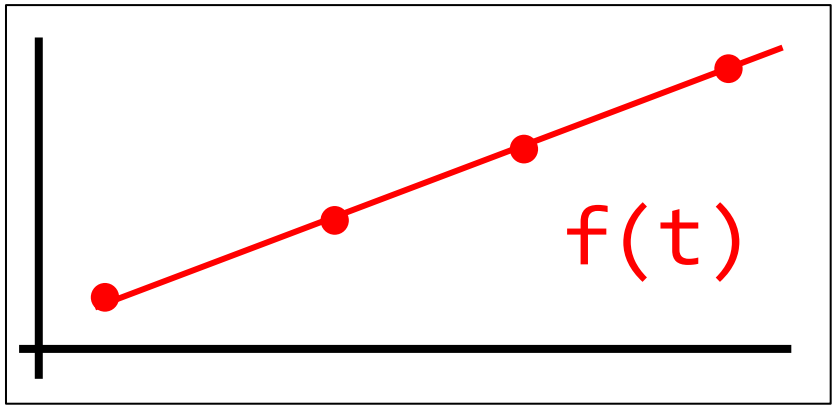




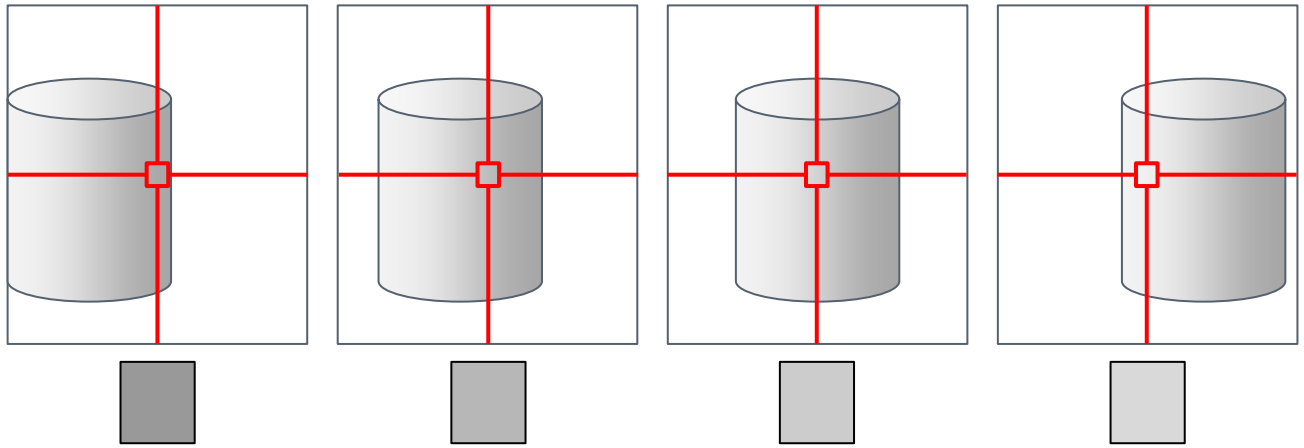
Optical flow - concept

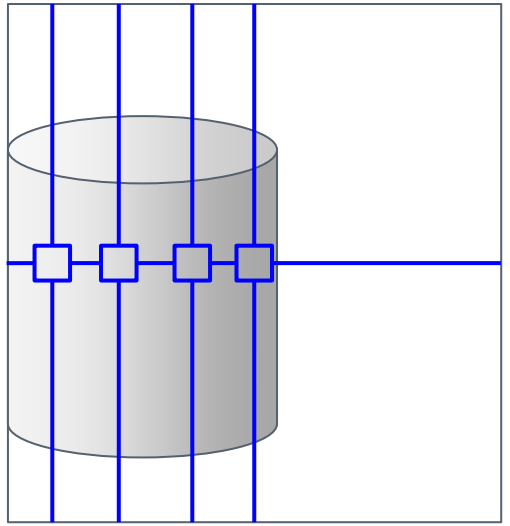


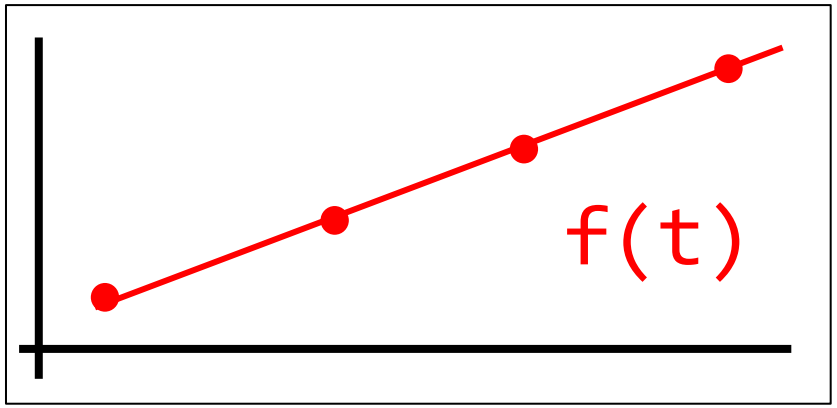




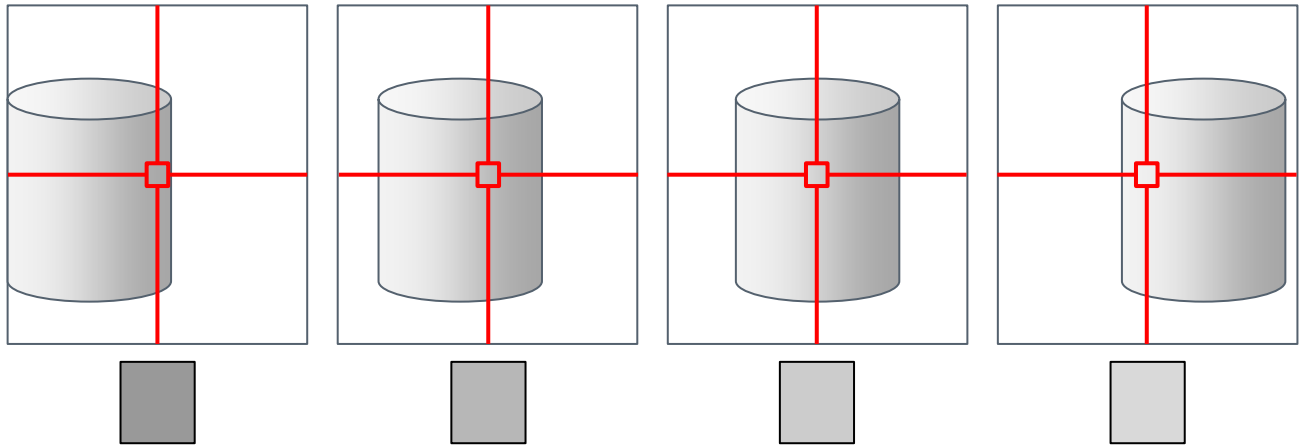
Optical flow - concept

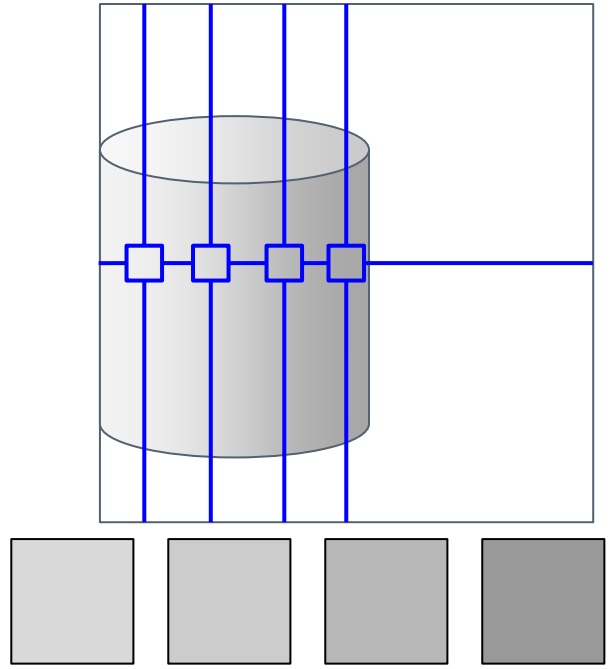


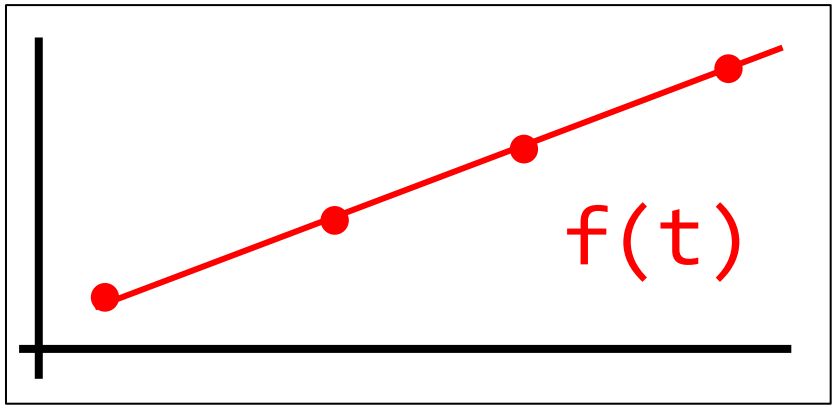




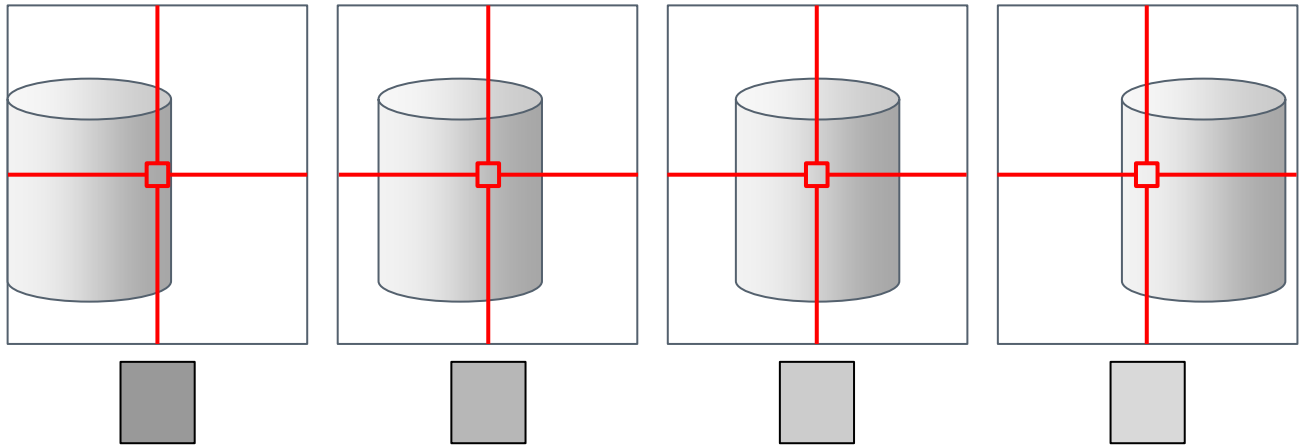
Optical flow - concept

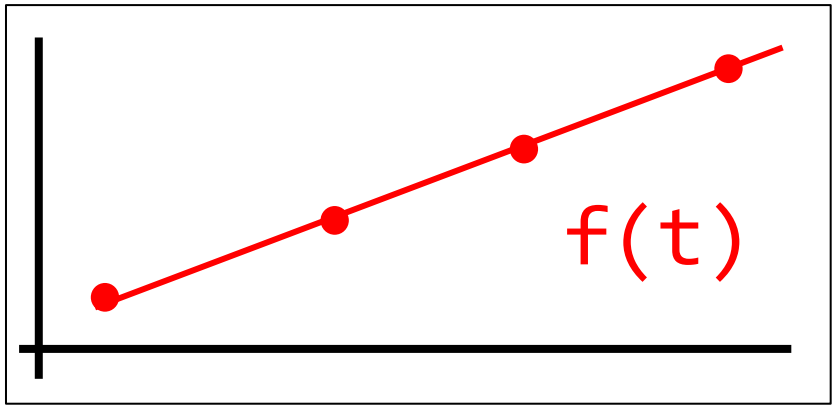


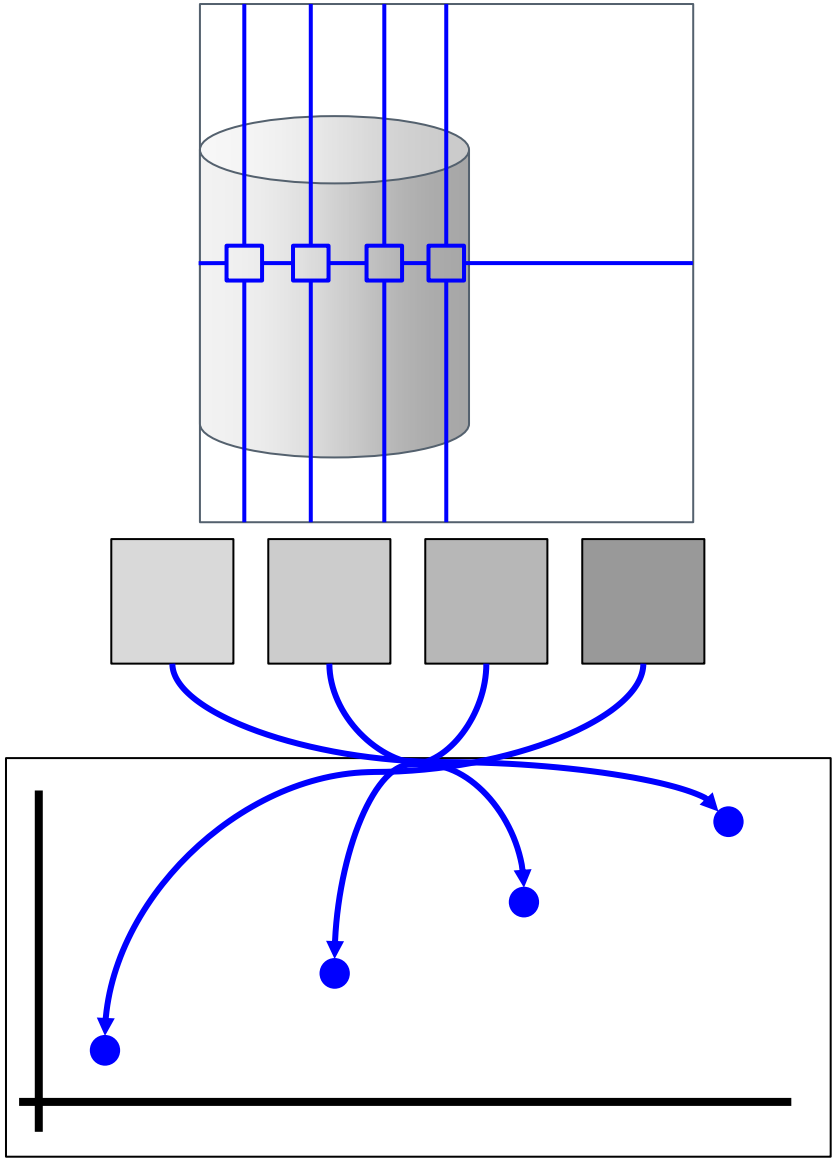




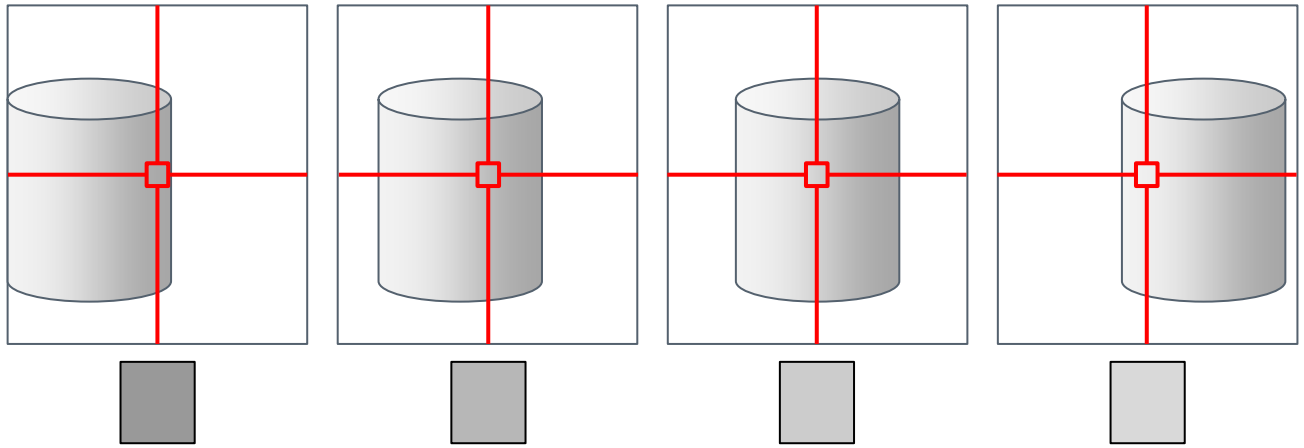
Optical flow - concept

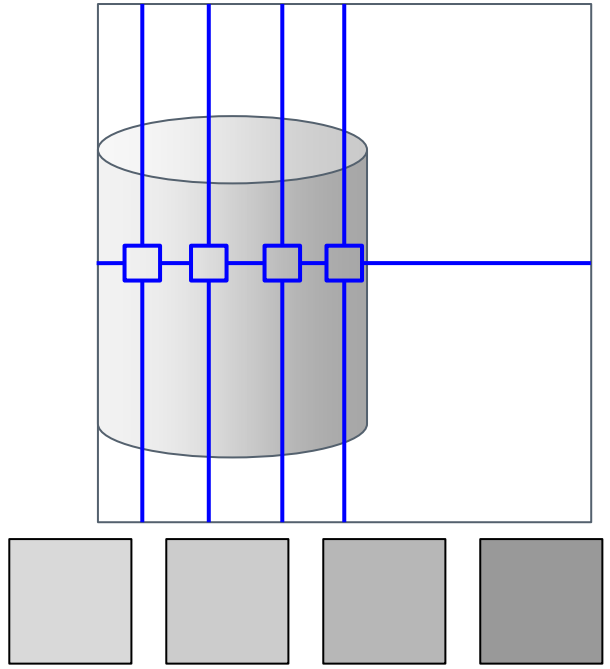


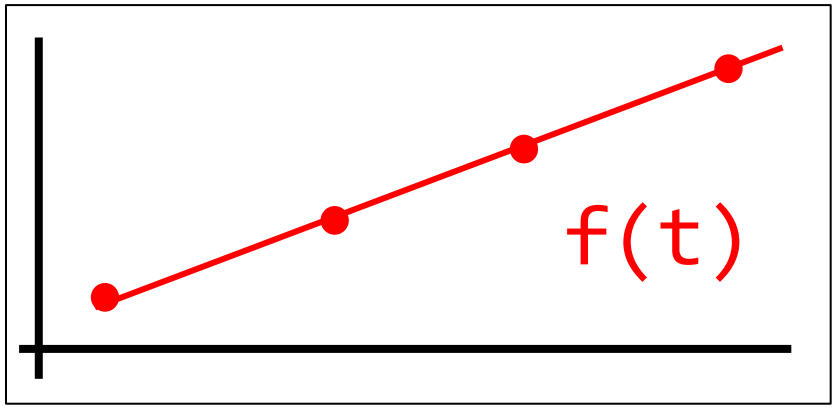


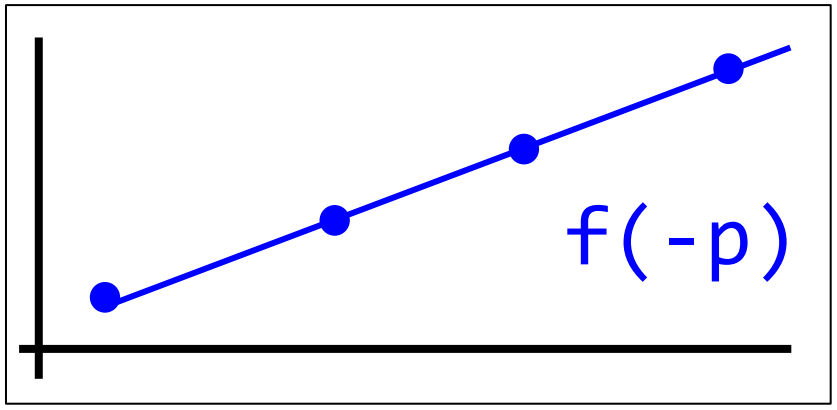


Optical flow - concept

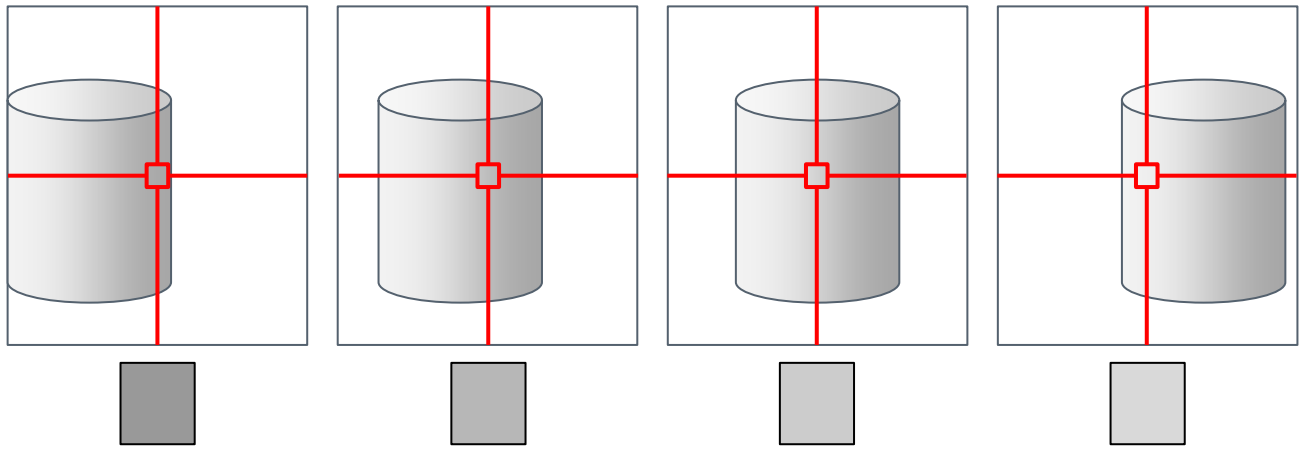


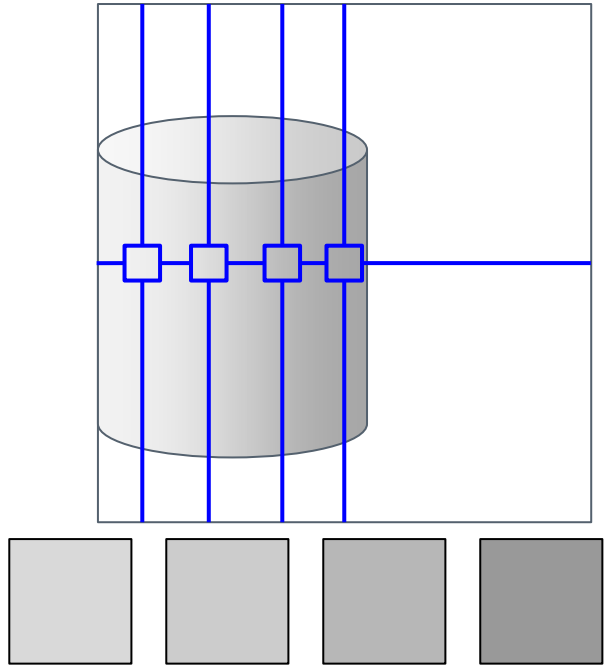




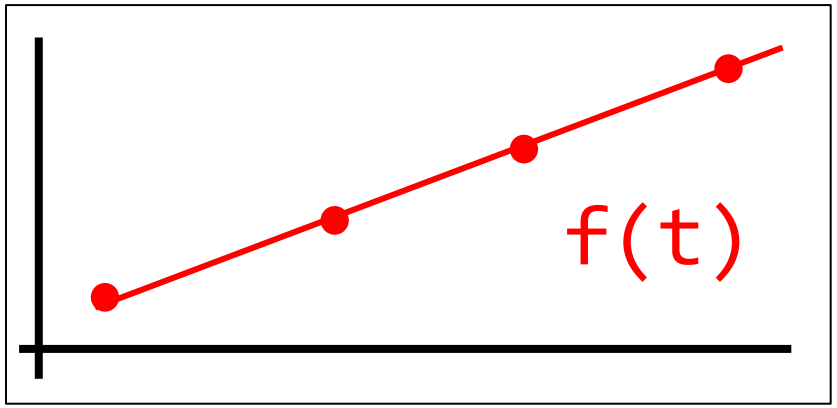


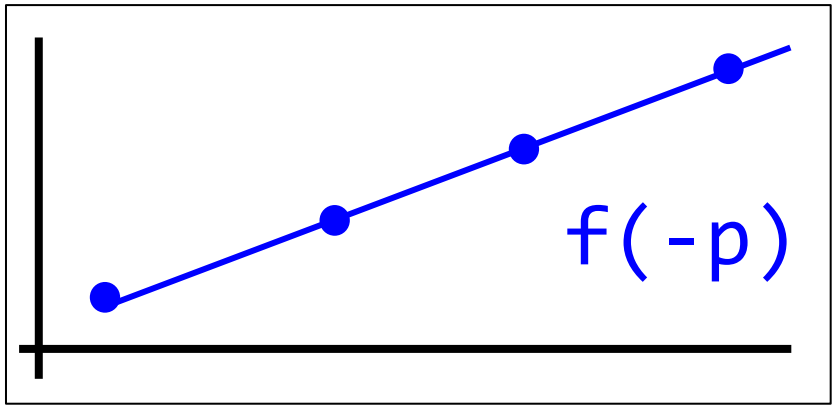
Optical flow - concept





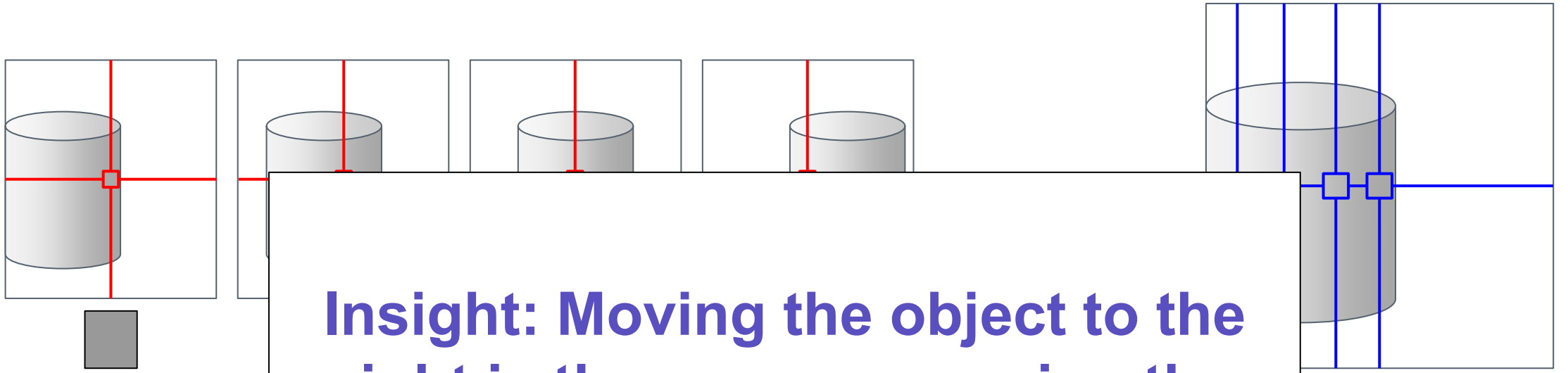
SAME!



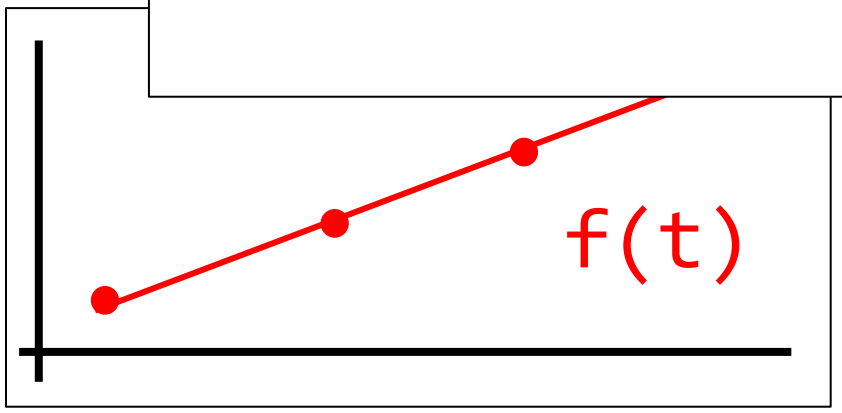


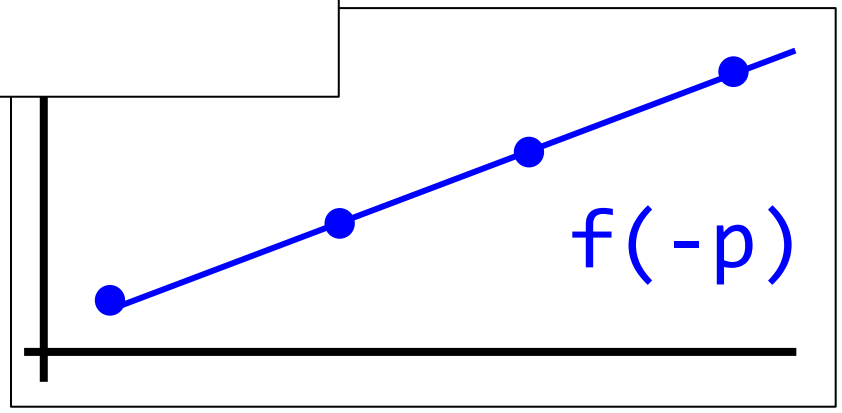
AALBORG UNIVERSITY
DENMARK

Optical flow - concept



Insight: Moving the object to the right is the same as moving the observed pixel to the left





Optical flow

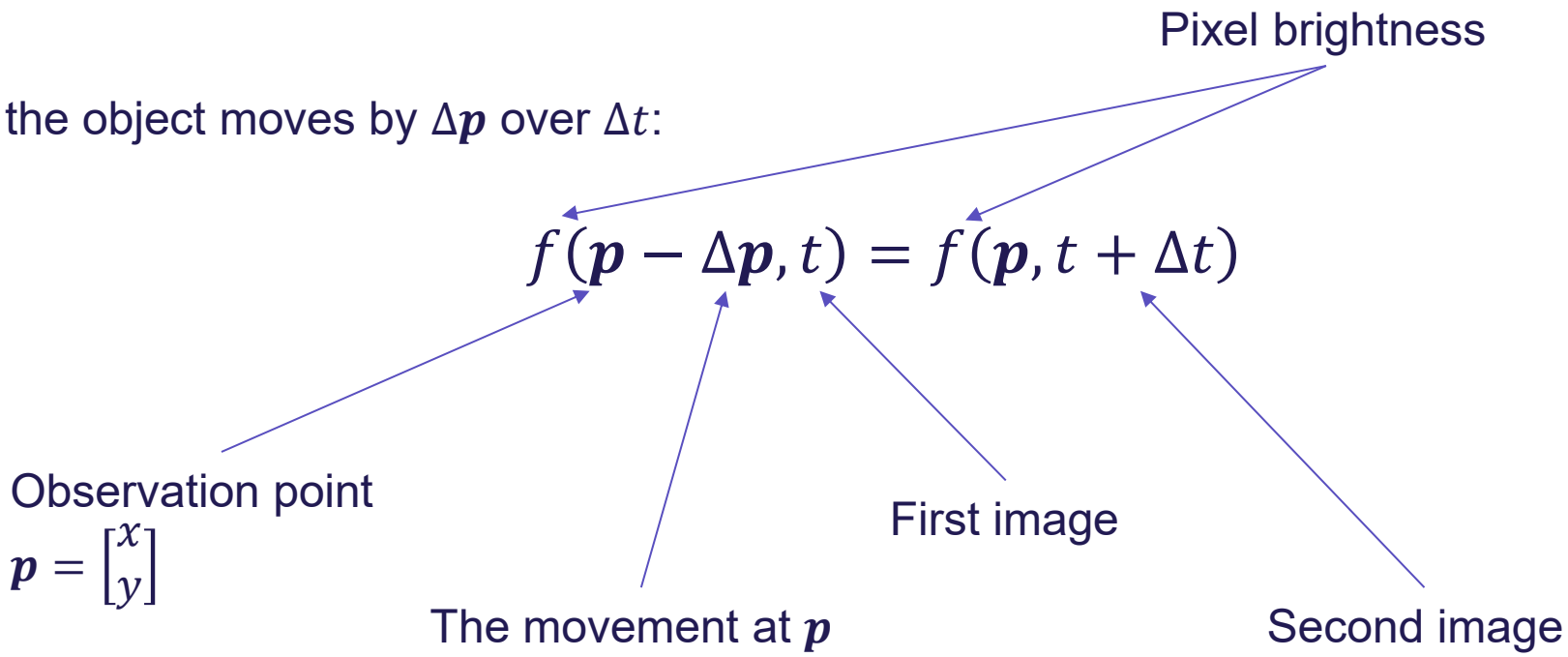
Can we use this relationship to compute optical flow?

Lucas-Kanade algorithm – one of the most well-known for optical flow

B.D. Lucas, T. Kanade, “An Iterative Image Registration Technique with an Application to Stereo Vision”, in Proceedings of Image Understanding Workshop, 1981, pp. 121-130.

Lucas-Kanade Optical Flow

Assuming the object moves by $\Delta \boldsymbol{p}$ over Δt:



Lucas-Kanade Optical Flow

Assuming the object moves by $\Delta \boldsymbol{p}$ over Δt:

$$f(\boldsymbol{p} - \Delta \boldsymbol{p}, t) = f(\boldsymbol{p}, t + \Delta t)$$

Approximate with a first-order Taylor expansion!

Lucas-Kanade Optical Flow

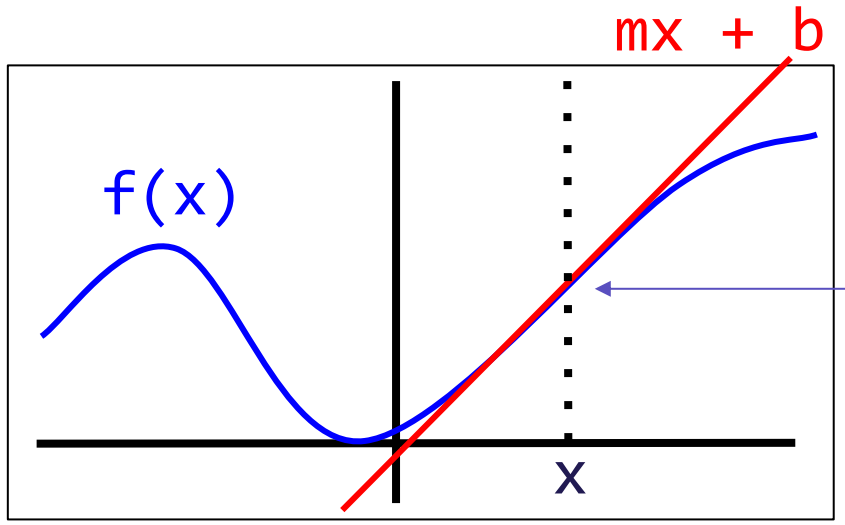
Approximate complex function with nth-order polynomial

$$f(x) \approx \sum_i a_i x^i$$

For 1st order $i = 1$

$f(x) \approx a_1 x^1 + a_0 x^0$

$f(x) \approx mx + b$



$f(x) = mx + b$ at x

Lucas-Kanade Optical Flow

Assuming the object moves by $\Delta \boldsymbol{p}$ over Δt:

$$f(\boldsymbol{p} - \Delta \boldsymbol{p}, t) = f(\boldsymbol{p}, t + \Delta t)$$

Approximate with a first-order Taylor expansion

$$m(\boldsymbol{p} - \Delta \boldsymbol{p}) + b \approx f(\boldsymbol{p}, t + \Delta t)$$

$$m\boldsymbol{p} - m\Delta \boldsymbol{p} + b \approx f(\boldsymbol{p}, t + \Delta t)$$

Lucas-Kanade Optical Flow

$$m\boldsymbol{p} - m\Delta\boldsymbol{p} + b \approx f(\boldsymbol{p}, t + \Delta t)$$

Subtract $f(\boldsymbol{p}, t)$ from both sides:

$$m\boldsymbol{p} - m\Delta\boldsymbol{p} + b - f(\boldsymbol{p}, t) \approx f(\boldsymbol{p}, t + \Delta t) - f(\boldsymbol{p}, t)$$

Recall $f(x, t) = mx + b$:

$$\cancel{m\boldsymbol{p}} - m\Delta\boldsymbol{p} + \cancel{b} - \cancel{(m\boldsymbol{p} + b)} \approx f(\boldsymbol{p}, t + \Delta t) - f(\boldsymbol{p}, t)$$

$$-m\Delta\boldsymbol{p} \approx f(\boldsymbol{p}, t + \Delta t) - f(\boldsymbol{p}, t)$$

What is this??

This is what we want to know

These we know

Lucas-Kanade Optical Flow

$$m\boldsymbol{p} - m\Delta\boldsymbol{p} + b \approx f(\boldsymbol{p}, t + \Delta t)$$

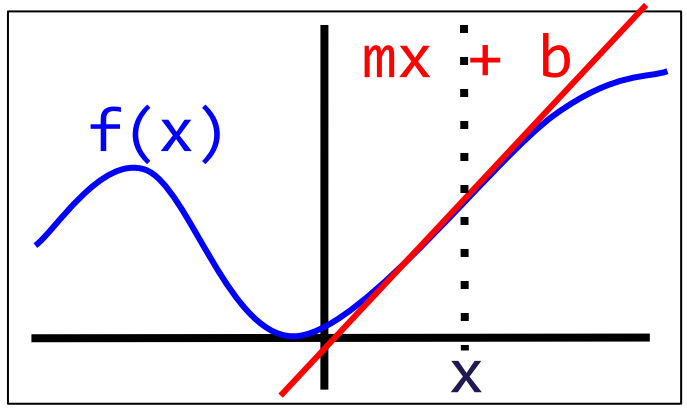
Subtract $f(\boldsymbol{p}, t)$ from both sides:

$$m\boldsymbol{p} - m\Delta\boldsymbol{p} + b - f(\boldsymbol{p}, t) \approx f(\boldsymbol{p}, t + \Delta t) - f(\boldsymbol{p}, t)$$

Recall $f(x, t) = mx + b$:

$$\cancel{m\boldsymbol{p}} - m\Delta\boldsymbol{p} + \cancel{b} - (\cancel{m\boldsymbol{p} + b}) \approx f(\boldsymbol{p}, t + \Delta t) - f(\boldsymbol{p}, t)$$

$$-m\Delta\boldsymbol{p} \approx f(\boldsymbol{p}, t + \Delta t) - f(\boldsymbol{p}, t)$$



Slope = derivative
Gradient!

$$m = d\boldsymbol{p} = \begin{bmatrix} dx & 0 \\ 0 & dy \end{bmatrix}$$

Lucas-Kanade Optical Flow

$$-m\Delta\boldsymbol{p} \approx f(\boldsymbol{p}, t+\Delta t) - f(\boldsymbol{p}, t)$$

Substitute $m = d\boldsymbol{p}$:

$$-d\boldsymbol{p}\Delta\boldsymbol{p} \approx f(\boldsymbol{p}, t+\Delta t) - f(\boldsymbol{p}, t)$$

Skip the vector notation:

$$-dx\Delta x - dy\Delta y \approx f\big((x,y), t+\Delta t\big) - f\big((x,y), t\big)$$

Clean up notation:

$$dx * u + dy * v = I_t[x,y] - I_{t+\Delta t}[x,y]$$

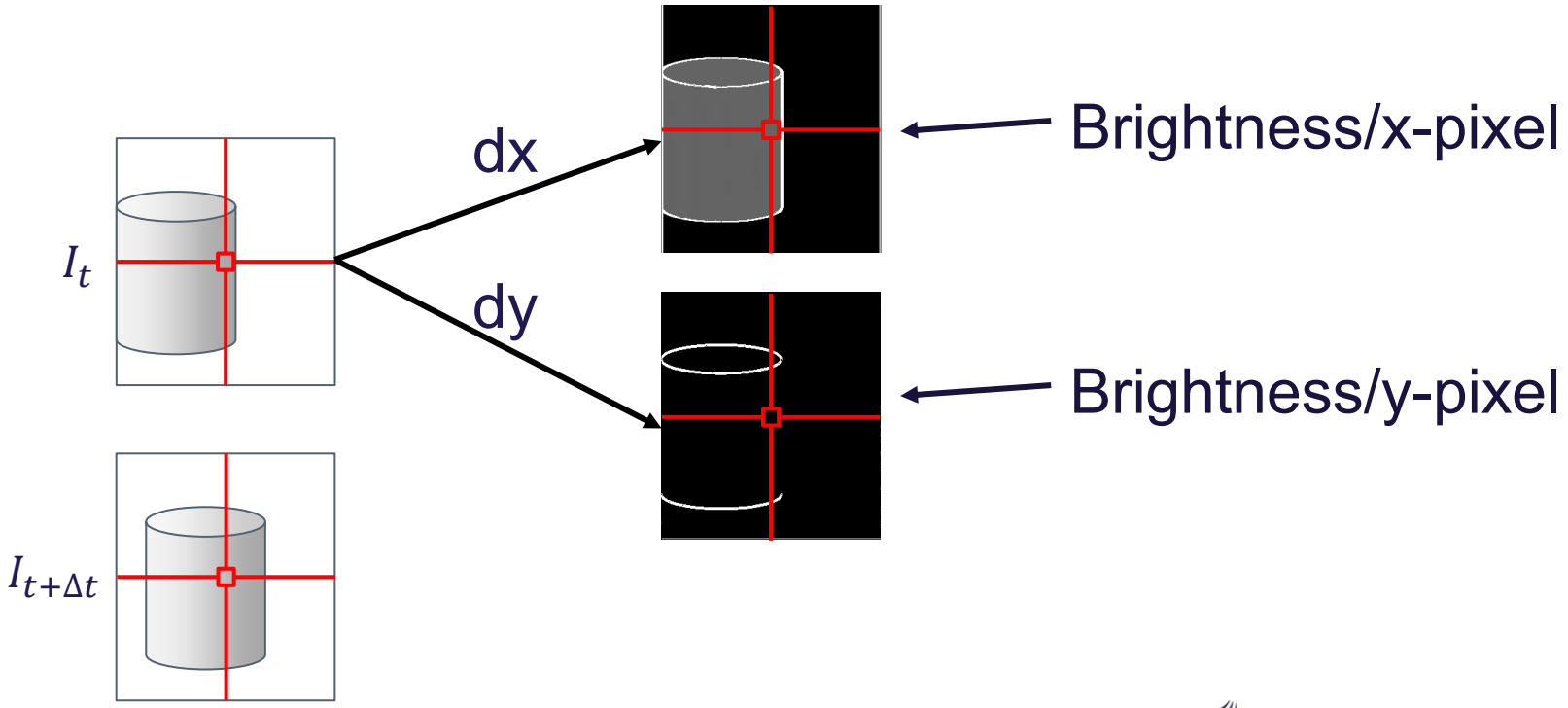
Solve for u and v:

1 equation, 2 unknowns

Impossible! ☹

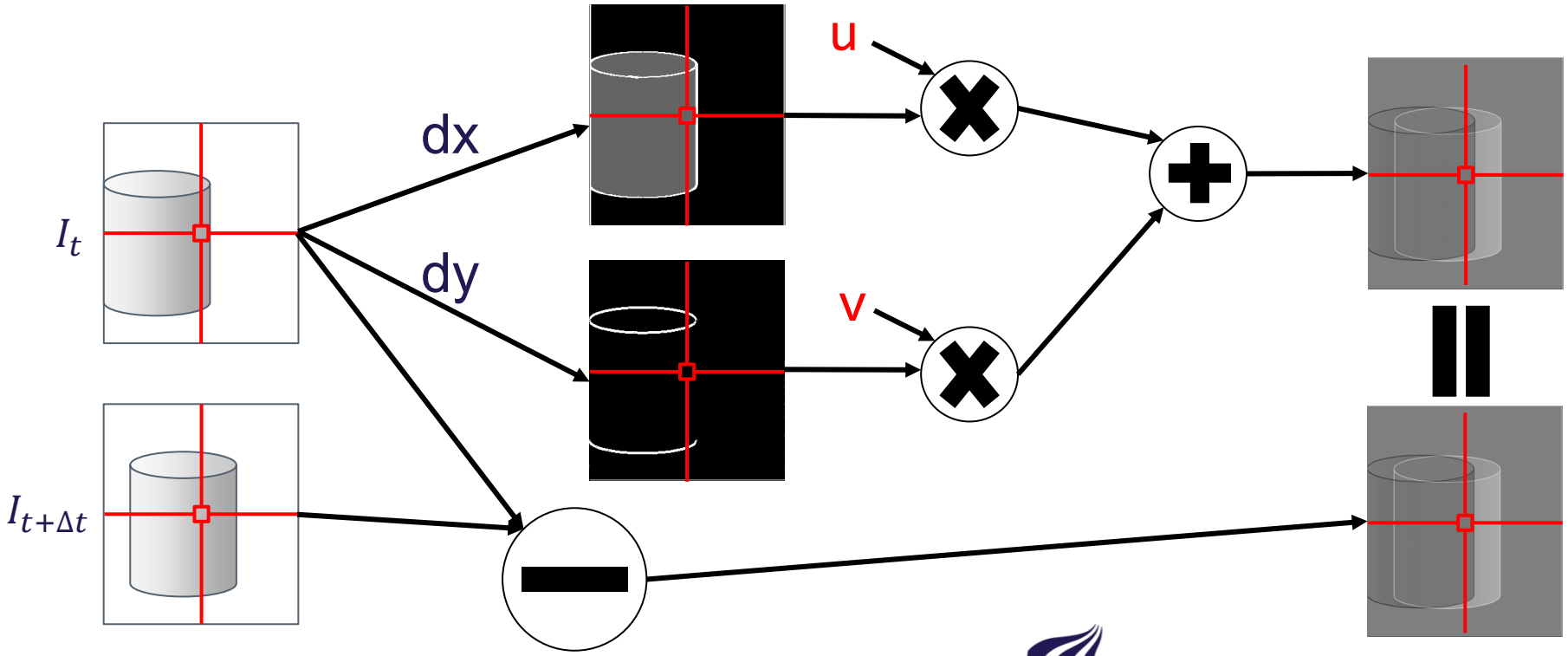
Lucas-Kanade Optical Flow

$$dx * u + dy * v = I_t[x, y] - I_{t+\Delta t}[x, y]$$



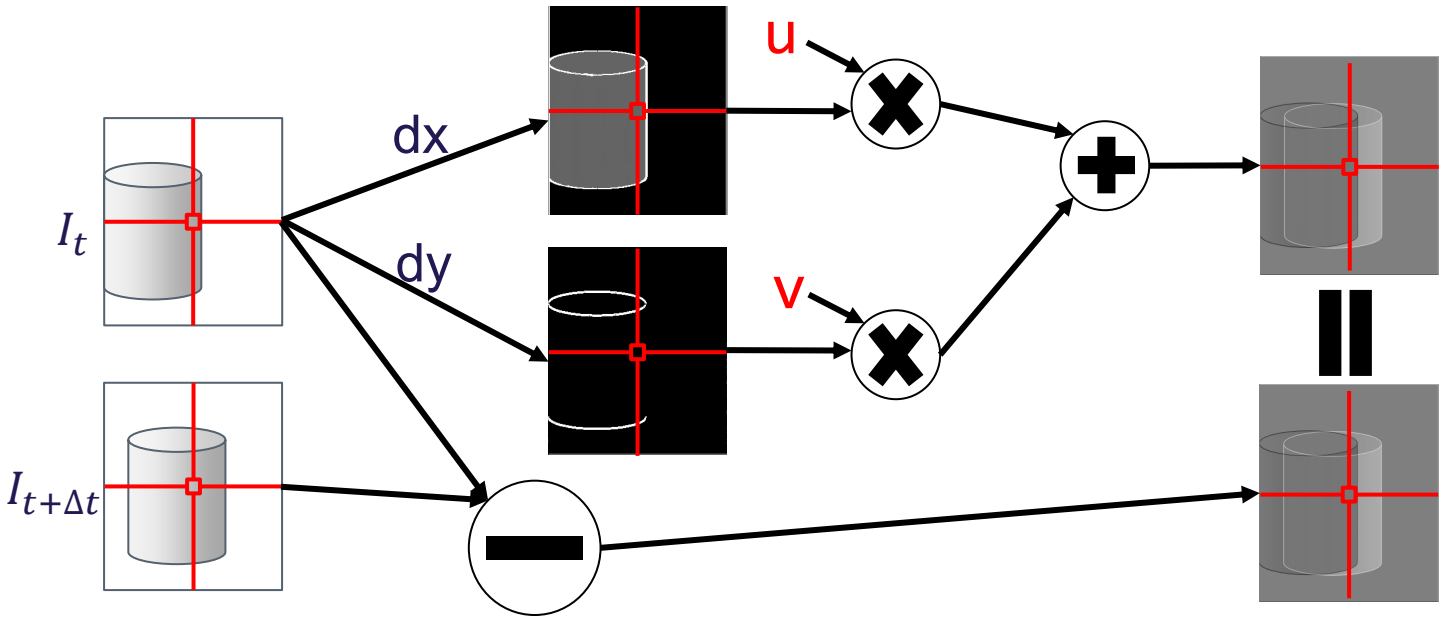
Lucas-Kanade Optical Flow

$$dx * u + dy * v = I_t[x, y] - I_{t+\Delta t}[x, y]$$



Lucas-Kanade Optical Flow

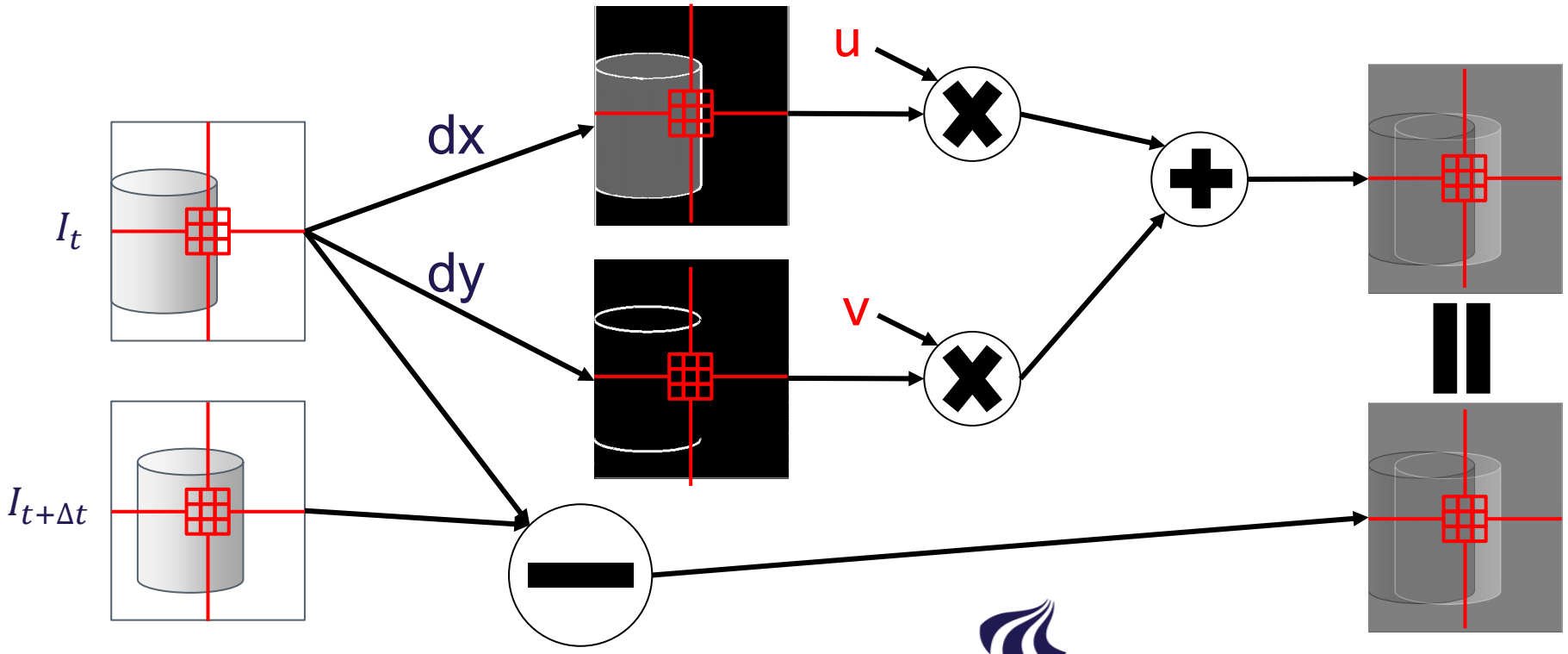
Underspecified: One equation, two unknowns



Idea: What if we assume neighbors move together? I.e., they have the same u and v.

Lucas-Kanade Optical Flow

$$dx * u + dy * v = I_t[x, y] - I_{t+\Delta t}[x, y]$$



Lucas-Kanade Optical Flow

$$dx_1 * u + dy_1 * v = I_t[x_1, y_1] - I_{t+\Delta t}[x_1, y_1]$$
$$dx_2 * u + dy_2 * v = I_t[x_2, y_2] - I_{t+\Delta t}[x_2, y_2]$$

$$\vdots$$

$$dx_9 * u + dy_9 * v = I_t[x_9, y_9] - I_{t+\Delta t}[x_9, y_9]$$

9 equations, 2 unknowns – overspecified!



Lucas-Kanade Optical Flow

Matrix notation:

$$S = \begin{bmatrix} dx_1 & dy_1 \\ dx_2 & dy_2 \\ \vdots & \vdots \\ dx_9 & dy_9 \end{bmatrix} \qquad \Delta\boldsymbol{p} = \begin{bmatrix} u \\ v \end{bmatrix} \qquad T = \begin{bmatrix} I_t[x_1, y_1] - I_{t+\Delta t}[x_1, y_1] \\ I_t[x_2, y_2] - I_{t+\Delta t}[x_2, y_2] \\ \vdots \\ I_t[x_2, y_2] - I_{t+\Delta t}[x_2, y_2] \end{bmatrix}$$

$$S\Delta\boldsymbol{p} = T$$

Least-squares solution:

$$\|S\Delta\boldsymbol{p} - T\|^2 = 0$$

$$\Delta\boldsymbol{p} = (S^T S)^{-1} S^T T$$

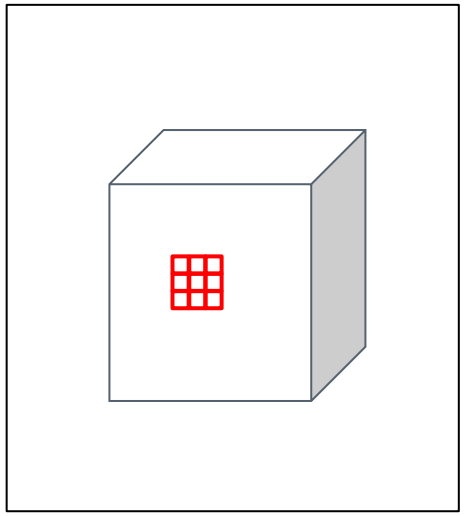
What if this isn't invertible??

E.g., no structure around (x,y)

Lucas-Kanade Optical Flow

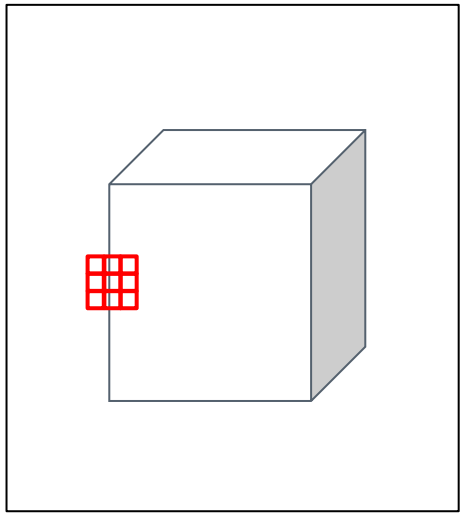
- How to pick (x,y)?

Bad



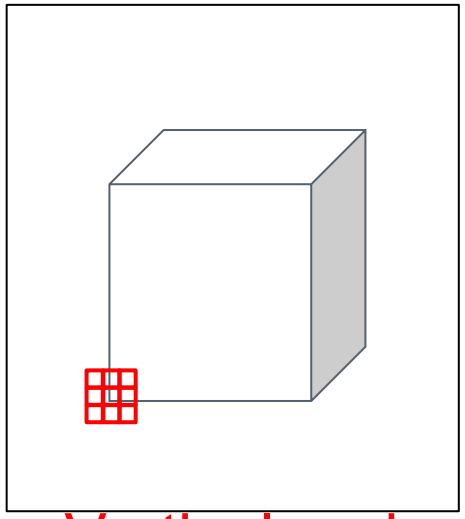
No gradient

Slightly better



Can’t detect vertical motion

Best



Vertical and horizontal motion

Lucas-Kanade Optical Flow

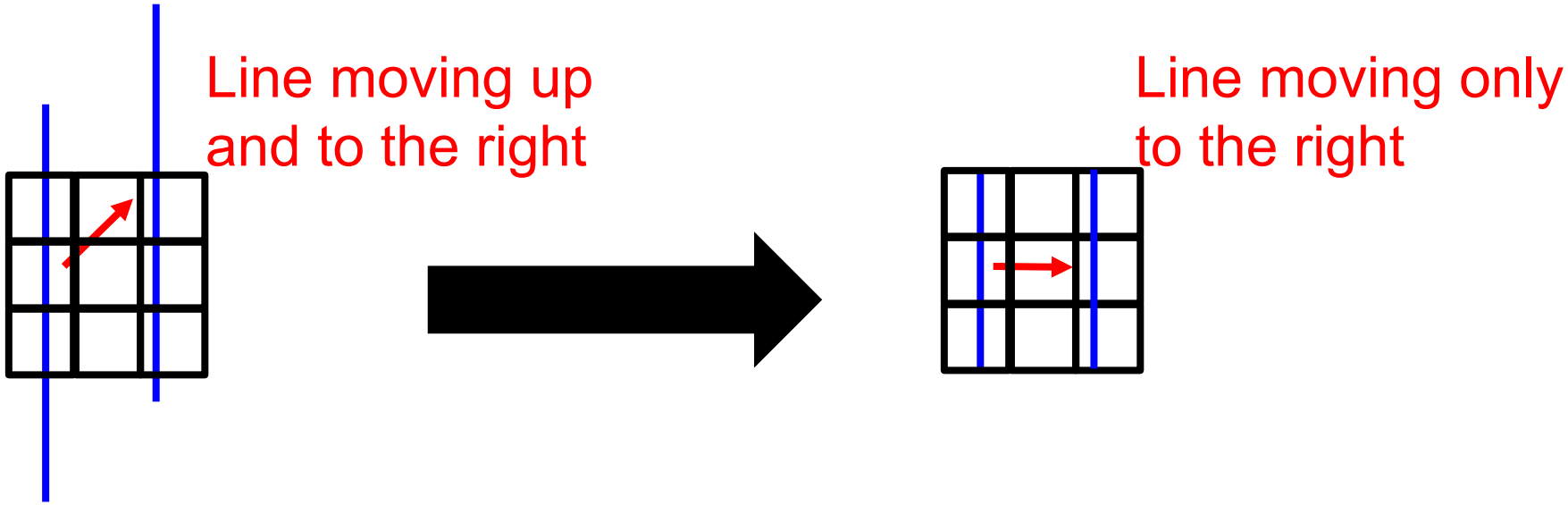
When does LK Optical Flow Fail?

- Lighting changes
- Large movement
- Specularities
- No good features
- Aperture problem



Lucas-Kanade Optical Flow

Aperture problem:



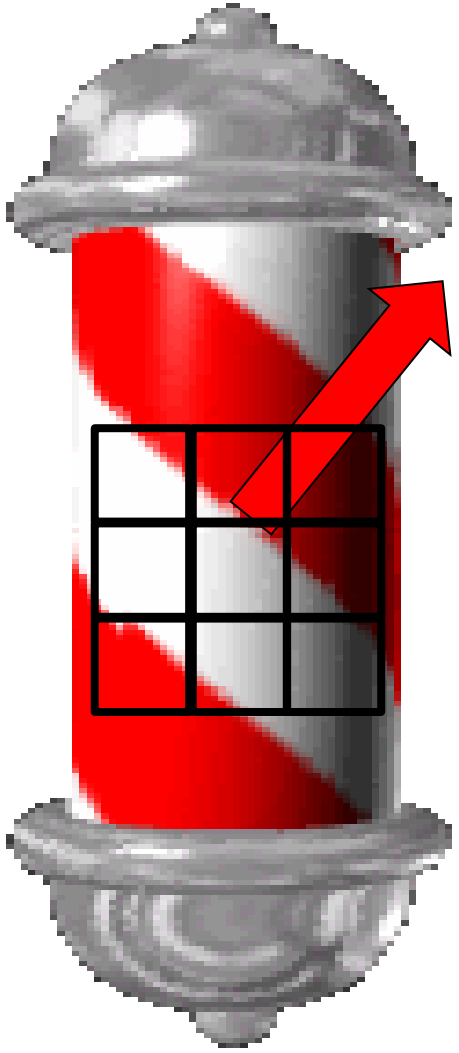
Lucas-Kanade Optical Flow

- Aperture problem – Barber pole illusion



Lucas-Kanade Optical Flow

- Aperture problem – Barber pole illusion



Apparent movement
to the up and right

Optical Flow

Sparse vs. Dense??

- Sparse: Compute flow only for specific features
- Dense: Compute flow for all pixels

Lucas-Kanade Optical Flow is technically a dense algorithm, but in practice only works on good feature points, i.e., it is sparse!

Dense optical flow:

Farnebäck, Gunnar. "Two-frame motion estimation based on polynomial expansion." *Scandinavian conference on Image analysis*. Springer, Berlin, Heidelberg, 2003.

Deep learning based implementation

- Check out RAFT
  - Pytorch implementation: https://docs.pytorch.org/vision/main/auto_examples/others/plot_optical_flow.html
  - Paper: https://arxiv.org/abs/2003.12039

Summary

Motion analysis in videos:

- Background subtraction
- Image differencing
- Optical flow

Next lecture:

- Tracking

